

ESP-IDF を Visual Studio Code で利用するまでの手順書

～ 開発環境構築からビルド・書き込み・動作確認まで ～

対象製品 / TARGET PRODUCT

CrowPanel Advanced 7inch
ESP32-P4 HMI AI Display 1024×600
(Elecrow社)

CHIP

ESP32-P4

FLASH

16 MB

PSRAM

32 MB

DISPLAY

1024×600

TOUCH

静電容量式

CONN.

Wi-Fi 6 / BT5.3

目次

- 0. はじめに（本書の目的・前提条件）

- 1. 全体の流れ（セットアップの全体像）

- 2. 事前準備（必要なハードウェア・環境）

- 3. Visual Studio Code のインストール

- 4. USBシリアルドライバのインストール

- 5. ESP-IDF拡張機能のインストール

- 6. ESP-IDF本体のセットアップ（Installation Manager）

- 7. サンプルプロジェクトの取得とターゲット設定

- 8. ビルド・書き込み・動作確認

- 9. トラブルシューティング

- 10. 付録：コマンド一覧・参考リンク

本書について

本書はElecrow社製「CrowPanel Advanced 7inch（ESP32-P4）」を対象に、Espressif公式のESP-IDF拡張機能を用いてVisual Studio Code上でビルド・書き込みができる状態になるまでの手順をまとめたものです。Elecrow社公式PDF「Advance HMI P4 7inch Course（V1.1）」を参考にしつつ、2026年8月時点の最新ツール（ESP-IDF Installation Manager等）に合わせて手順を整理しています。

本書の目的

本書は、ESP32-P4搭載の7インチHMIディスプレイ「CrowPanel Advanced 7inch」を用いて、Visual Studio Code（以下VS Code）上でESP-IDFによる開発（ビルド・書き込み・シリアルモニタでの動作確認）ができる状態にするまでの手順を、初めての方でも迷わず進められるように整理したものです。

対象読者

- ESP-IDFやVS Codeを初めて使う方
- Arduino IDEでのESP32開発は経験があるが、ESP-IDF（Espressif公式のネイティブ開発フレームワーク）は初めての方
- CrowPanel Advanced 7inch（ESP32-P4）で公式サンプルを動かしてみたい方

前提条件

- ☐ Windows 10/11、macOS、またはLinux（Ubuntu等）が動作するPC
- ☐ インターネット接続（初回セットアップ時に数GB規模のダウンロードが発生します）
- ☐ PCの管理者権限（ドライバインストール・ツールインストールに必要）
- ☐ CrowPanel Advanced 7inch 本体、およびUSB Type-Cケーブル

なぜArduino IDEではなくESP-IDFなのか

ESP32-P4はRISC-Vアーキテクチャを採用した比較的新しいチップです。Arduino IDEやPlatformIOでも開発は可能ですが、2026年8月時点でESP32-P4はPlatformIOの公式サポート対象外となっています。Espressif社公式のESP-IDF+VS Code拡張機能の組み合わせが、最も安定してESP32-P4のフル機能（MIPI-DSI、PSRAM、USB等）を扱える開発環境です。

1

全体の流れ（セットアップの全体像）

OVERVIEW

セットアップは大きく分けて次の**6つのフェーズ**で進みます。各フェーズの詳細は2章以降で解説します。

① 事前準備

ハードウェア・PC環境の確認

② VS Code導入

エディタ本体のインストール

③ USBドライバ

CH341ドライバの導入

④ 拡張機能導入

ESP-IDF拡張機能のインストール

⑤ ESP-IDF構築

Installation Managerで本体一式を導入

⑥ 動作確認

サンプル取得→ビルド→書き込み
→モニタ

フェーズ	内容	所要時間目安	ポイント
①	事前準備	5分	電源はUSB給電のみだと不足する場合あり
②	VS Codeインストール	5分	公式サイトからダウンロードするだけ
③	USBドライバ	5～10分	CH341チップ用。macOSは既知の不具合に注意
④	ESP-IDF拡張機能	2分	VS Code内のマーケットプレイスから導入
⑤	ESP-IDF本体構築	10～45分	回線速度に依存。Easy Installationで最新安定版（例：v5.5.5）を導入
⑥	ビルド～動作確認	10～15分	BOOTボタン操作でダウンロードモードへ

 動画解説をご覧の方へ

本書と対になるスライド（全体像図解）をご用意しています。動画内ではフェーズ①～⑥の順に画面をデモしながら解説していますので、本書は「実際に手を動かす際の詳細手順書」としてご活用ください。

2

事前準備

PREREQUISITES

2-1. 必要なハードウェア

- ☐ CrowPanel Advanced 7inch（ESP32-P4）本体
- ☐ USB Type-Cケーブル（データ通信対応のもの。充電専用ケーブルは不可）
- ☐ （推奨）USB-C ACアダプタ、または十分な給電能力を持つUSBハブ

⚠ 電源に関する注意

本製品は7インチIPSディスプレイのバックライトを含め、動作時に約8～10Wの電力を必要とします。PCのUSBポートからの給電のみでは電力が不足し、画面が真っ暗になったり、再起動を繰り返したりすることがあります。安定して動作しない場合は、別途USB-C ACアダプタから給電するか、セルフパワー式のUSBハブを利用してください。

2-2. PC環境

項目	要件
OS	Windows 10/11、macOS（Intel/Apple Silicon）、Linux（Ubuntu 20.04以降 等）
ディスク空き容量	10GB以上（ESP-IDFツールチェーン一式のため）
メモリ	8GB以上を推奨
ネットワーク	ブロードバンド環境推奨（初回に数GBダウンロード）
権限	ソフトウェア・ドライバをインストールできる管理者権限

2-3. 対象製品の主なスペック（参考）

項目	仕様
メインチップ	ESP32-P4（RISC-V デュアルコア、高性能コア最大400MHz + 低消費電力コア）
無線通信	ESP32-C6-MINI-1モジュール搭載（Wi-Fi 6 / Bluetooth 5.3）
Flash	16MB（NOR Flash）
PSRAM	32MB（Octal SPI, 200MHz）※画面フレームバッファのため必須
ディスプレイ	7.0インチ IPS、1024×600、静電容量式タッチ（GT911コントローラ）

USBポート	USB Type-C × 2 (① UART0 : CH341チップ経由 ② USB 2.0 Device : ESP32-P4 ネイティブ)
その他	microSDスロット、11ピンGPIOヘッダ、Crowtail(I2C/UART)、スピーカー/マイク、MIPI-CSIカメラ対応(オプション)

3

Visual Studio Code のインストール

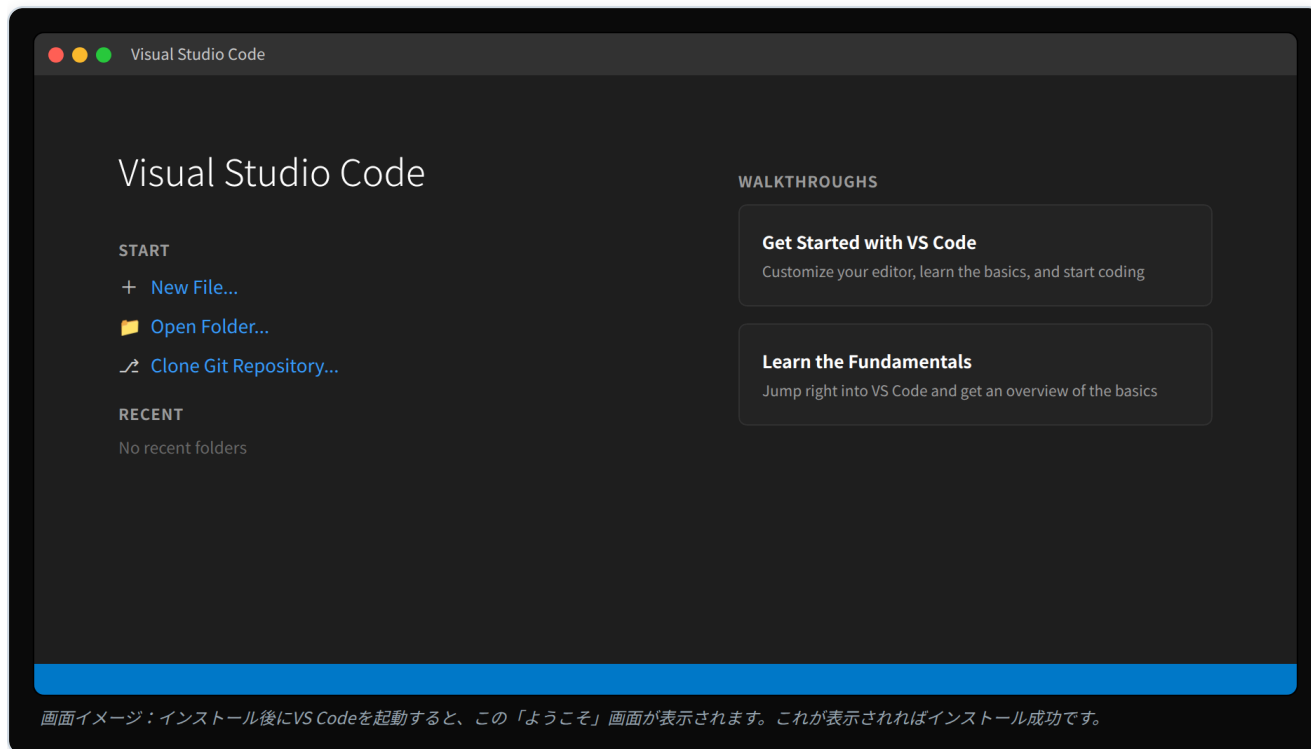
INSTALL VS CODE

3-1. インストーラのダウンロード

1. 公式サイト <https://code.visualstudio.com/> にアクセスします。
2. 使用しているOS（Windows / macOS / Linux）に対応したインストーラをダウンロードします。
3. ダウンロードしたインストーラを実行し、画面の指示に従ってインストールします（基本的にすべて既定設定のままで問題ありません）。

3-2. 起動確認

1. インストール完了後、VS Codeを起動します。
2. ウェルカム画面が表示されれば、インストールは正常に完了しています。



💡 Tips

すでにVS Codeをインストール済みの場合は、この章はスキップして構いません。ただし、拡張機能のインストールでエラーが出る場合は、一度最新バージョンに更新（ヘルプ>更新の確認）してから進めることをおすすめします。

4

USBシリアルドライバのインストール

USB-SERIAL DRIVER (CH341)

CrowPanel Advanced 7inch の「UART0」ポート（プログラム書き込み用ポート）には、WCH（沁恒微电子）社製のUSB-シリアル変換チップ**CH341**が使用されています。PCから本ポート経由でシリアル通信を行うには、専用ドライバのインストールが必要です。

4-1. Windowsの場合

1. WCH公式ダウンロードページ https://www.wch-ic.com/downloads/CH341SER_ZIP.html から「CH341SER.ZIP」をダウンロードします。
2. ZIPを展開し、含まれるインストーラ（SETUP.EXE等）を管理者権限で実行します。
3. 「INSTALL」 ボタンを押してドライバをインストールします。
4. インストール後、念のためPCを再起動します。

期待される結果：スタートメニューから「デバイスマネージャー」を開き、「ポート (COMとLPT)」を展開すると、次のように「USB-SERIAL CH340」というポートが表示されます。これが表示されていれば、ドライバのインストールは成功です（本製品をPCに接続した状態で確認してください）。



4-2. macOSの場合

1. WCH公式ダウンロードページ https://www.wch-ic.com/downloads/CH34XSER_MAC_ZIP.html からドライバをダウンロードし、インストールします。
2. 「システム設定 > プライバシーとセキュリティ」で、ドライバの実行許可（カーネル拡張機能の許可）を求められた場合は許可してください。

期待される結果：「ターミナル」アプリを開き、次のコマンドを実行します。

```
ls /dev/tty.*
```

本製品をUSBケーブルでPCに接続した状態で、一覧に `/dev/tty.wchusbserial****` のような名前が表示されていれば、ドライバは正しく認識されています。

⚠ macOSでの既知の不具合

macOS Tahoe (26系) 以降の一部環境では、CH341ドライバに起因してUART0ポートが正しく認識されない、あるいは通信が不安定になる既知の不具合が報告されています。UART0ポートで認識・書き込みができない場合は、もう一方のポートである「USB 2.0 Device」ポート (ESP32-P4がネイティブに備えるUSBシリアル/JTAG機能を利用したポートで、追加ドライバ不要) に接続を切り替えて試してください。

4-3. Linuxの場合

1. 多くのディストリビューションでは、CH34x系ドライバがカーネルに標準搭載されているため、追加インストールなしでポートが認識されます。
2. 認識されない場合のみ https://www.wch-ic.com/downloads/CH341SER_LINUX_ZIP.html からドライバを入手してください。
3. また、ユーザーがシリアルポートにアクセスできるよう、`dialout` グループへの追加が必要な場合があります (`sudo usermod -aG dialout $USER` 実行後に再ログイン)。

期待される結果：ターミナルで次のコマンドを実行し、本製品をUSBケーブルでPCに接続した状態で `/dev/ttyUSB0` のような名前が表示されていれば認識成功です。

```
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

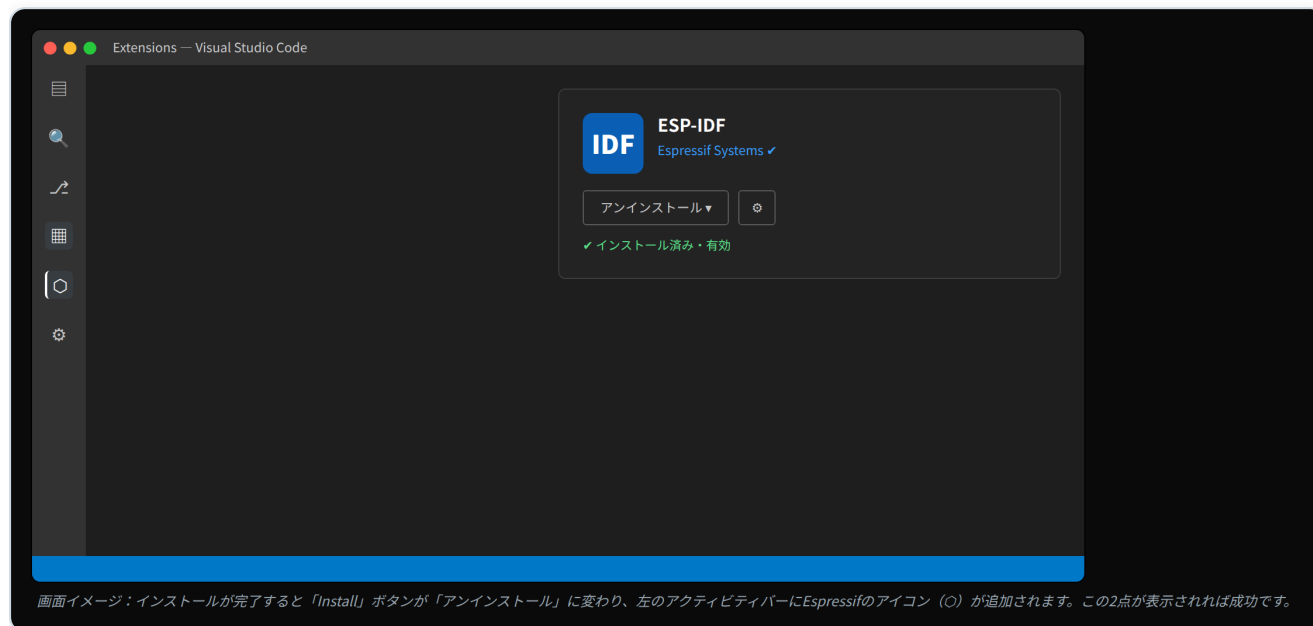
5

ESP-IDF拡張機能のインストール

INSTALL THE VS CODE EXTENSION

1. VS Codeを起動します。
2. 左側アクティビティバーの「拡張機能」アイコン（またはショートカット `Ctrl+Shift+X` / macOSは `Cmd+Shift+X`）をクリックします。
3. 検索ボックスに `ESP-IDF` と入力します。
4. 発行元が「**Espressif Systems**」の拡張機能「**ESP-IDF**」（識別子：`espressif.esp-idf-extension`）を選択し、「Install」をクリックします。
5. インストールが完了すると、左側アクティビティバーにEspressifのアイコンが追加されます。

期待される結果：「Install」ボタンが「アンインストール」に変わり、左側のアクティビティバーに六角形のEspressifアイコンが表示されます。



🚧 この時点ではまだビルドできません

拡張機能自体は「VS Code上でESP-IDFを操作するためのUI・コマンド群」を提供するものです。実際にビルドを行うには、次章でESP-IDF本体（コンパイラ・ツールチェーン・Pythonライブラリ等一式）をインストールする必要があります。

ESP-IDF拡張機能には「ESP-IDF Installation Manager (EIM)」が統合されており、これを使うことでコンパイラやPython環境、ESP-IDF本体一式をまとめて自動インストールできます。各手順の下に「実行後に表示されるはずの画面」のイメージ図を添えていますので、実際の操作結果と見比べながら進めてください（画面はイメージ図であり、OSやバージョンにより実際の表示と細部が異なる場合があります）。

この章で行う2つのこと（全体像）

この章は手順が多く感じられますが、やっていることは大きく2つだけです。迷ったら、今どの段階にいるかをここで確認してください。

- ① **インストーラを準備する（6-1～6-3）**：「EIM」という自動インストール用の小さなプログラム自体を、まずPCに用意します。
- ② **ESP-IDF本体をインストールする（6-4～6-8）**：EIM自身の画面を進み、コンパイラ・Python環境・ESP-IDF本体（合計数GB）を実際にダウンロード＆インストールします。本書では「**Easy Installation**」という、あらかじめ動作確認済みの一式をまとめてインストールする方法を使います。画面数が少なく、ほとんどは既定のまま進めるだけです。ここが一番時間がかかります。

インストールしたESP-IDFを「このプロジェクトで使う」とVS Codeに教える作業（**Select Current ESP-IDF Version**）は、次の7章でプロジェクトのフォルダーを開いた直後に行います。この設定はプロジェクトのフォルダーごとに保存される仕組みのため、フォルダーを開く前には実行できません。この章では、まずインストール自体を完了させることに集中してください。

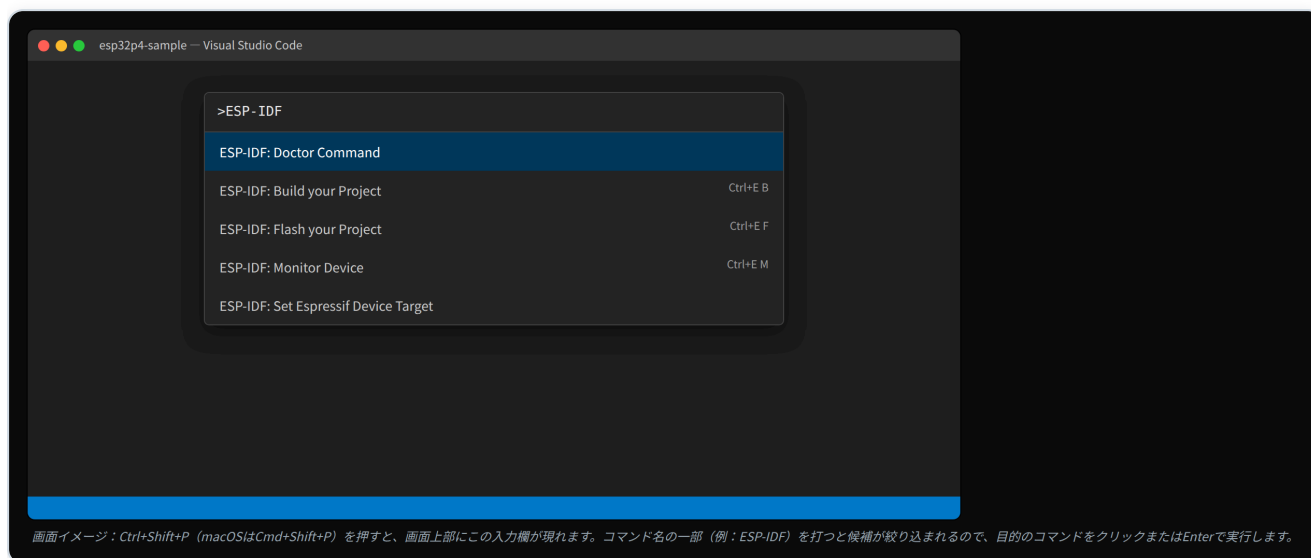
EIMの画面は英語表示・バージョンにより見た目が変わります

ここから先の6-4～6-12は、「EIM (ESP-IDF Installation Manager)」という別プログラム自身の画面です。VS Code拡張機能とは異なり日本語化されておらず、すべて英語表示です。また、EIM自体のバージョンが上がると画面デザインが変更されることがあります（本書は執筆時点のWindows版 EIM v0.18.0の実際の画面をもとに作成しています。macOSでは表示が異なる場合があります）。表示が本書と多少違っていても、基本的には画面の指示に沿って「Continue」を押していけば問題ありません。

用語解説：「コマンドパレット」とは

本章以降、「コマンドパレットから **ESP-IDF: ∞** を実行します」という表現が繰り返し出てきます。これはVS Codeの中心となる操作窓口で、メニューを辿らなくてもキーボードだけでほぼ全ての操作を検索・実行できる機能です。

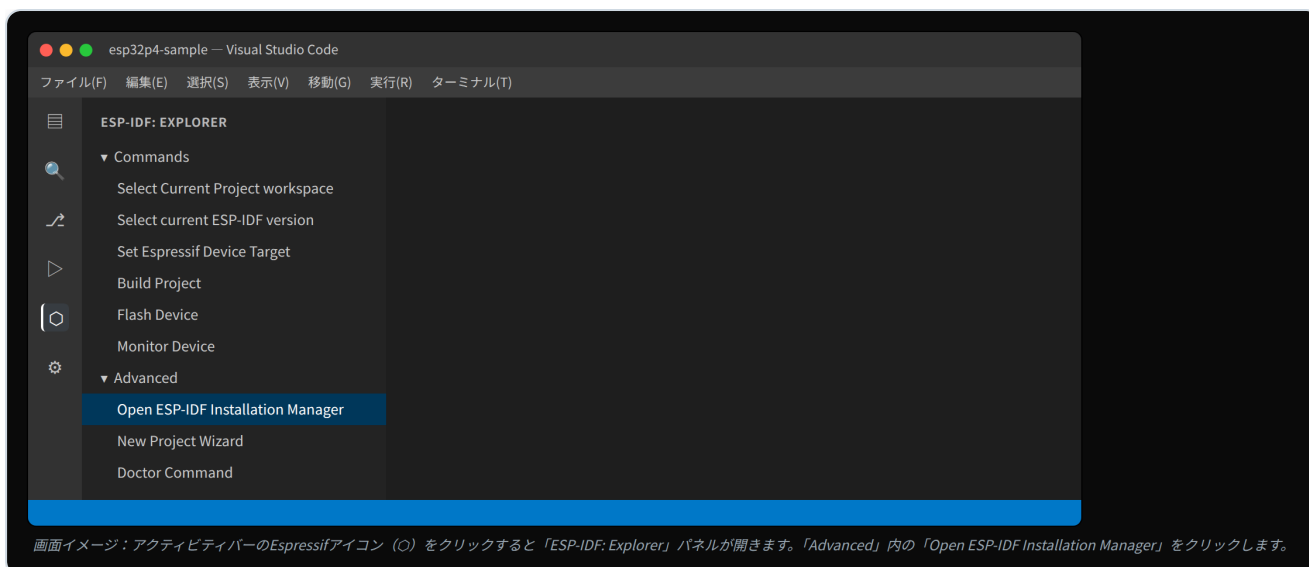
- **起動方法**： **Ctrl+Shift+P**（macOSは **Cmd+Shift+P**）を押します。
- **使い方**：画面上部に検索ボックスが表示されるので、実行したいコマンド名の一部（例： **ESP-IDF: Doctor**）を入力します。該当するコマンドが一覧に絞り込まれるので、クリックするか、ハイライトされた状態でEnterキーを押します。
- **期待される結果**：入力に応じて候補が一覧表示され、選んだ項目を実行すると検索ボックスは閉じ、選んだコマンドの処理（ビルドの開始、パネルが開く、等）が始まります。何も起きない・候補が0件の場合は、入力したコマンド名が正しいか確認してください。



画面イメージ：Ctrl+Shift+P（macOSはCmd+Shift+P）を押すと、画面上部にこの入力欄が現れます。コマンド名の一部（例：ESP-IDF）を打つと候補が絞り込まれるので、目的のコマンドをクリックまたはEnterで実行します。

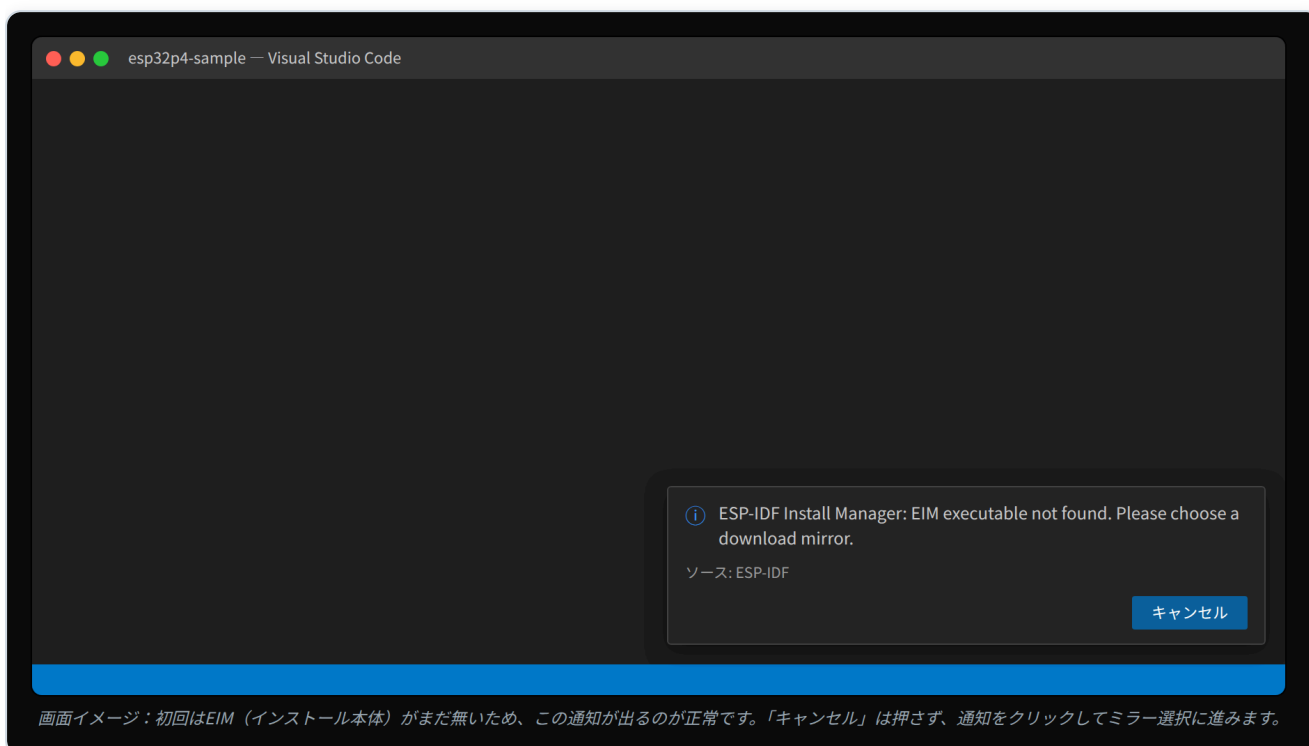
6-1. ESP-IDF: Explorerパネルを開く

1. VS Code左側のアクティビティバーにあるEspressifのアイコン（六角形のマーク）をクリックします。
2. 「ESP-IDF: Explorer」パネルが開き、「Commands」「Advanced」などの項目一覧が表示されます。
3. 「Advanced」を展開し、「**Open ESP-IDF Installation Manager**」をクリックします。



6-2. Installation Managerの起動

クリック直後、初回は右下に次のような通知が表示されます。「キャンセル」は押さず、通知をクリックして次のミラー選択に進んでください。



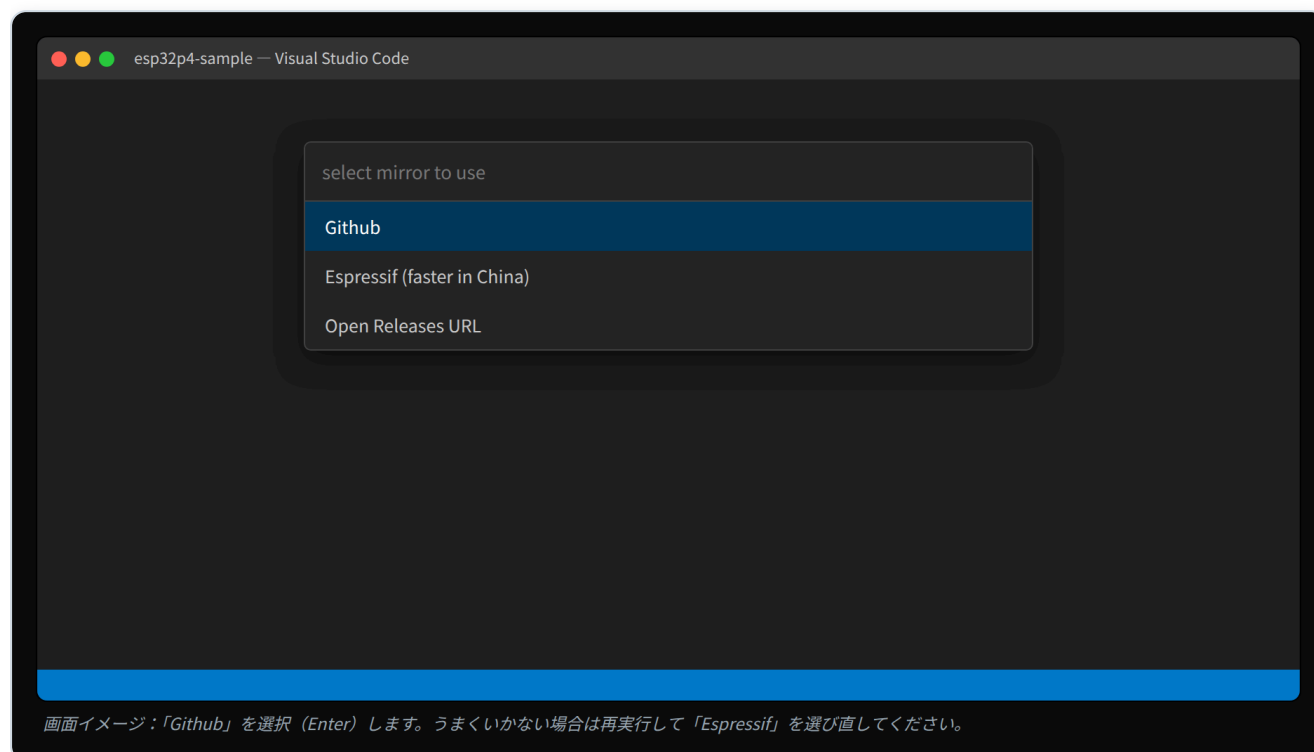
なぜこの通知が出るのか

ESP-IDF拡張機能自体には、実際にPython・ツールチェーン・ESP-IDF本体をダウンロード&インストールする処理は含まれていません。その実処理は「EIM (ESP-IDF Installation Manager)」という別の小さなプログラム（実行ファイル）が担当しています。

そのため拡張機能は、「Open ESP-IDF Installation Manager」が実行されるとまず「EIMの実行ファイルはPC上にあるか？」を確認します。多くの場合、初めてこの操作をしたPCにはEIMがまだ存在しないため、「EIM executable not found (EIMの実行ファイルが見つかりません)。どこからダウンロードしますか？」という、この通知が表示される、という流れです。つまりこれは不具合ではなく、「**本体をインストールする前に、まずインストーラ自身を用意する**」ための正常な確認です。

6-3. ダウンロード元（ミラー）の選択

「select mirror to use」というドロップダウンが表示されます。これはURLへのリンクではなく、「EIMの実行ファイルをどこからダウンロードするか」を選ぶ選択肢です。ここでは一番上の「Github」を選択してください（クリック、またはEnterキー）。



「Github」「Espressif」「Open Releases URL」は何を選んでいるのか

表示される3つの選択肢は、いずれも同じEIMの実行ファイルの「入手先（配布元）」の違いです。

- **Github** … Espressif社がGitHub上で公開している配布ページ（Releases）から自動ダウンロードします。世界的に見て最も一般的で、日本国内からも通常は問題なく使えるため、まずはこれを選びます。
- **Espressif（faster in China）** … 同じファイルをEspressif社自身のサーバーから取得します。中国など、GitHubへの接続が遅い・不安定な地域向けの代替ミラーです。
- **Open Releases URL** … 自動ダウンロードを行わず、配布ページをブラウザで開くだけの選択肢です。自動取得がどうしても失敗する場合に、手動でファイルを保存するために使います。

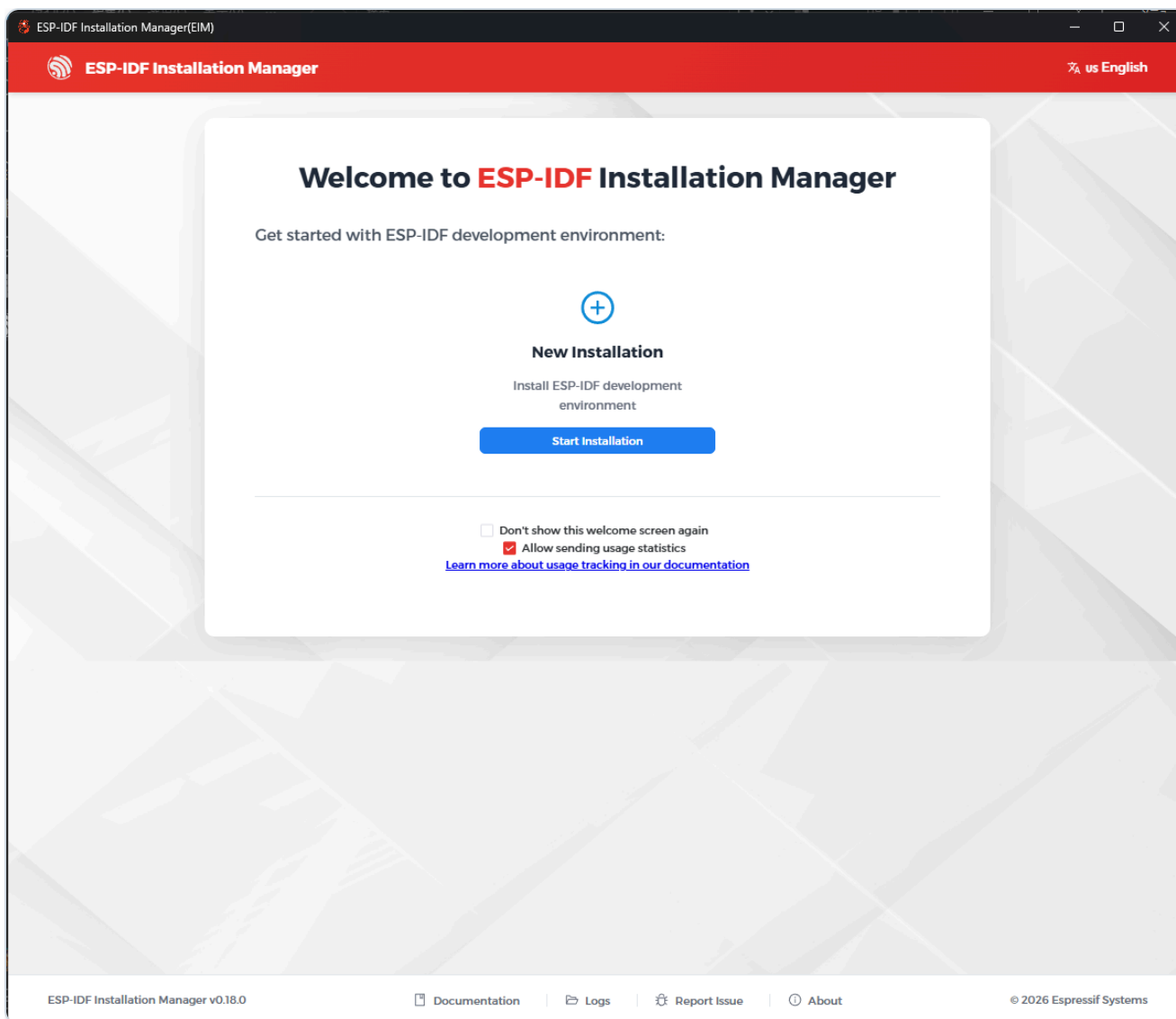
「Github」を選ぶと、拡張機能が裏側で該当ファイルを自動ダウンロードし、完了次第そのままEIM（ESP-IDF Installation Manager）本体が起動します。ブラウザでどこかのサイトを開くわけではないため、リンクらしい見た目でも実際にはクリックしても画面遷移は起きません。

Githubでうまくいかない場合

ネットワーク環境によっては「Github」からのダウンロードがタイムアウトすることがあります。その場合はもう一度「Open ESP-IDF Installation Manager」を実行し直し、今度は「Espressif（faster in China）」を選択してみてください。

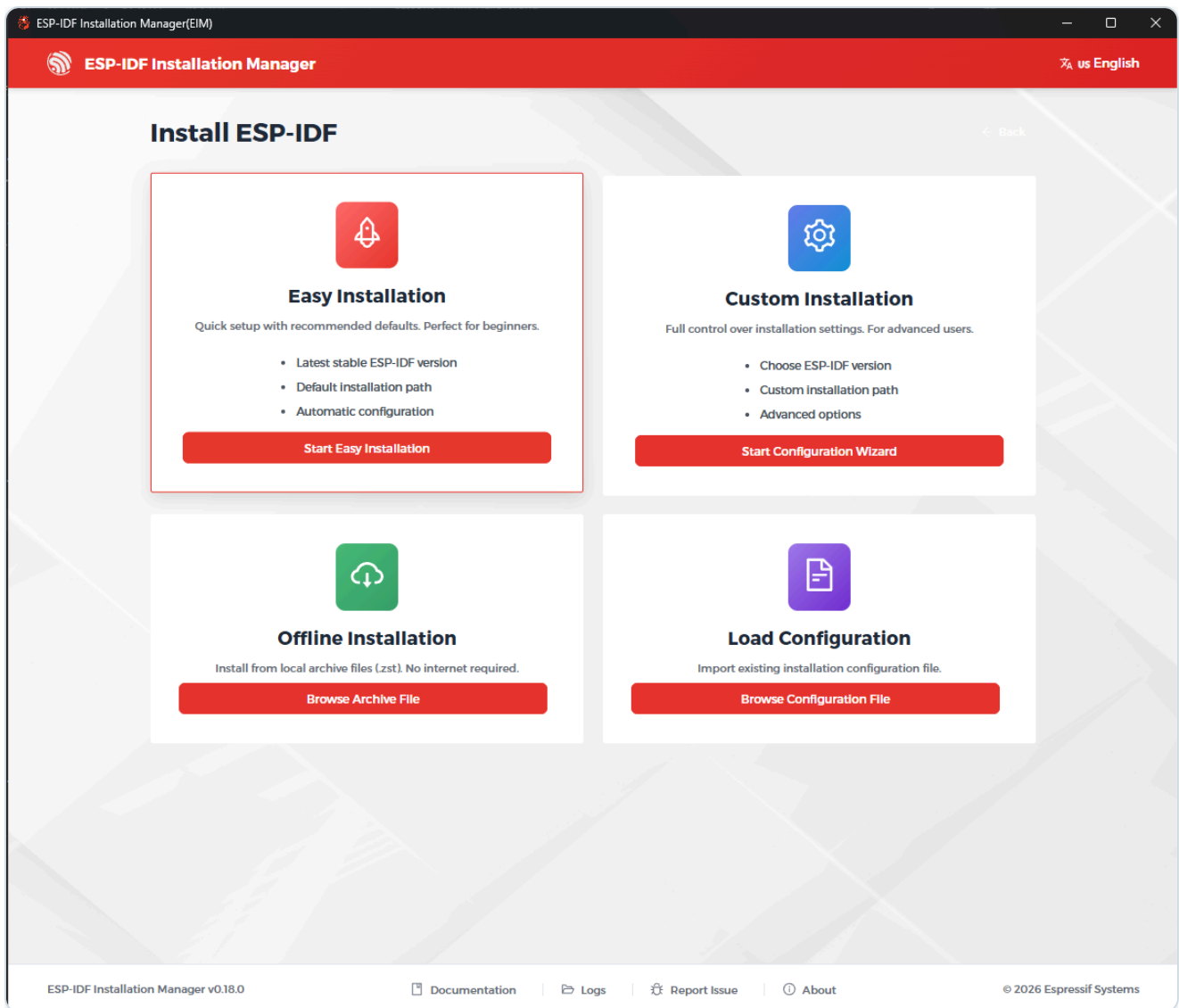
6-4. EIMのウェルカム画面

ダウンロードが終わるとEIM（ESP-IDF Installation Manager）が自動的に起動し、次のウェルカム画面が表示されます。「**Start Installation**」をクリックします。



6-5. インストール方法の選択（Easy Installationを選ぶ）

「Easy Installation」「Custom Installation」「Offline Installation」「Load Configuration」の4種類が表示されます。本書では、あらかじめ動作確認済みの一式をまとめて導入できる「**Easy Installation**」の「**Start Easy Installation**」をクリックしてください。ターゲットチップの個別選択やミラーの選択といった細かい画面がなく、最短の手順でインストールが完了します。

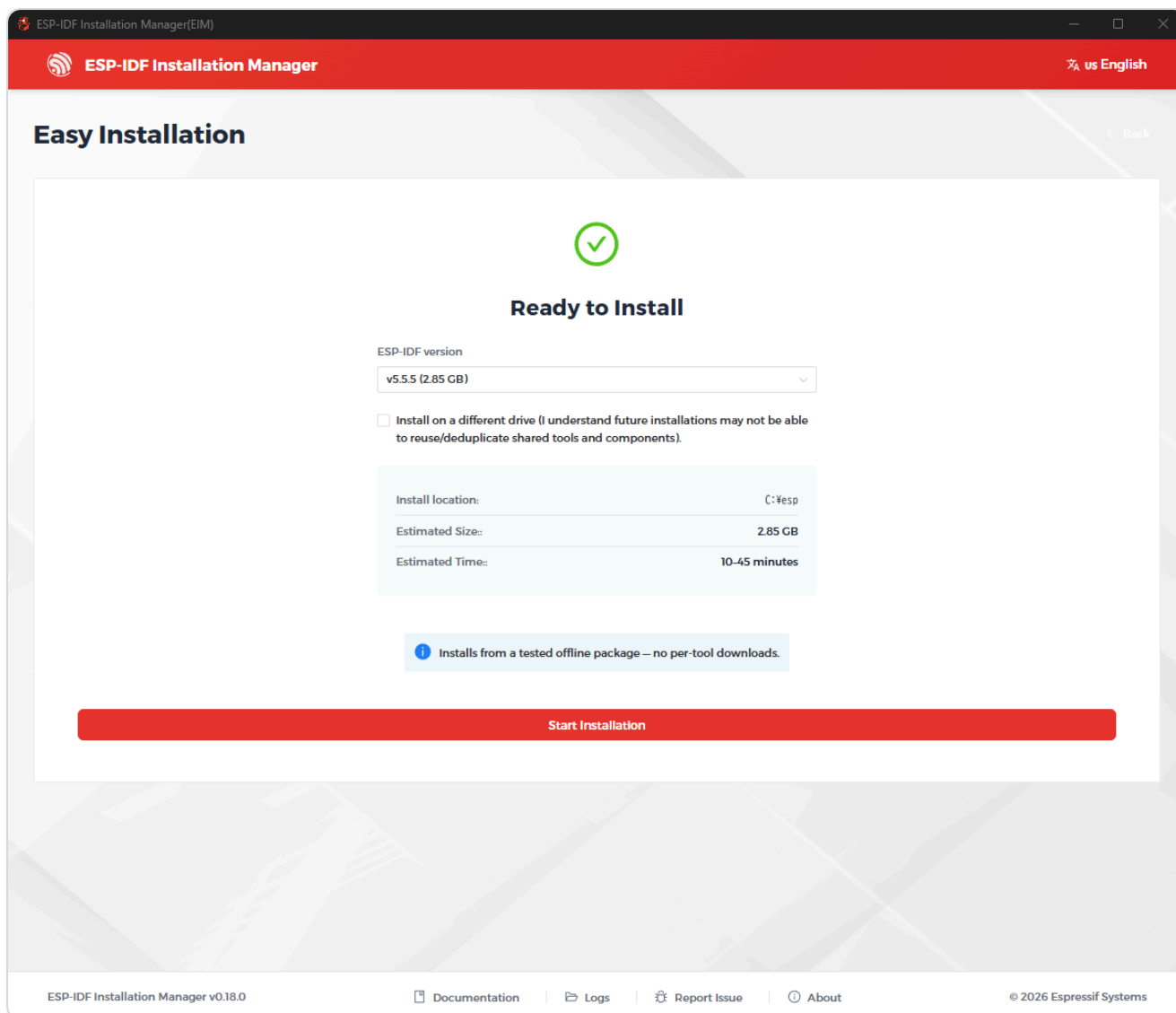


特定のESP-IDFバージョンを指定したい場合

Easy Installationは既定で最新の安定版（本書執筆時点では **v5.5.5**）を導入します。次の6-6の画面にはバージョンを選ぶドロップダウンもありますが、選べるのは基本的に比較的新しい安定版のみです。Elecrow公式サンプルが前提とする **v5.4.2** のように、あえて古い特定バージョンを指定したい場合は、Custom Installationを使う**10-3（付録）**の手順を参照してください。まずは本章の手順で最新安定版を導入し、動作確認を進めることを推奨します。

6-6. インストール内容の確認（Ready to Install）

導入されるESP-IDFのバージョン、インストール先フォルダ、想定容量・所要時間が表示されます。内容を確認し、「**Start Installation**」をクリックしてください。

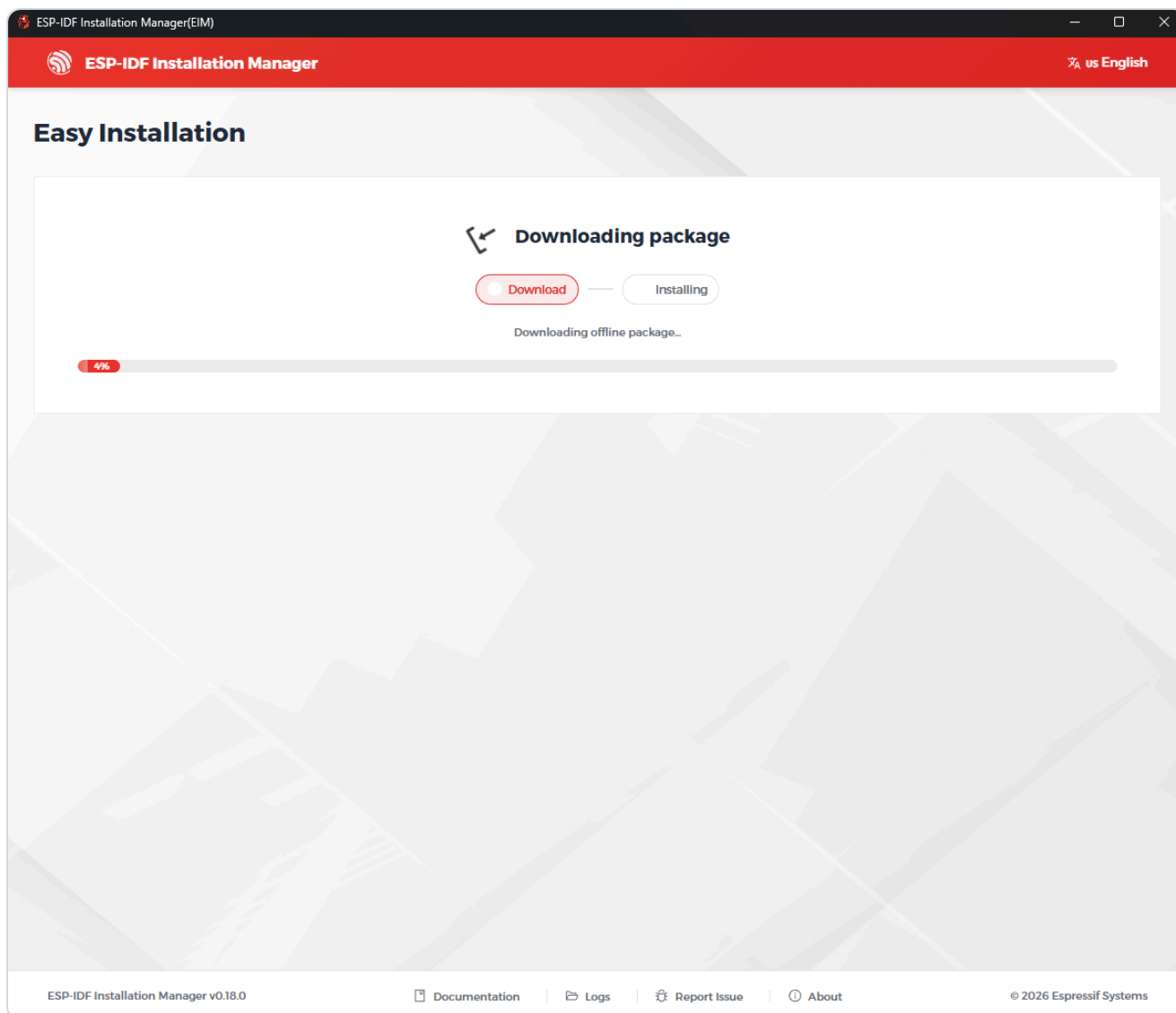


- **ESP-IDF version** : 導入されるバージョンです。本書では既定の **v5.5.5 (2.85 GB)** のまま進めます。
- **Install location** : ESP-IDF本体のインストール先です。既定の **C:\esp** のような短いパスであれば、そのまま問題ありません。「Install on a different drive」にはチェックを入れないでください。
- **Estimated Size / Estimated Time** : ダウンロード容量と所要時間の目安です。回線速度によって**10～45分程度**かかります。

※ 「Installs from a tested offline package — no per-tool downloads.」という案内の通り、Easy Installationはツールごとに個別ダウンロードするのではなく、あらかじめ動作確認済みの一式（オフラインパッケージ）をまとめて取得・展開します。本書の検証では、この方式の方がCustom Installationより展開エラーが起きにくいことを確認しています（詳しくは9章のトラブルシューティングを参照）。

6-7. インストールの実行

6-6の画面で「Start Installation」をクリックすると、「Download（オフラインパッケージのダウンロード）」→「Installing（展開・設定）」の2段階でインストールが進みます。回線速度によって**合計10～45分程度**かかります。

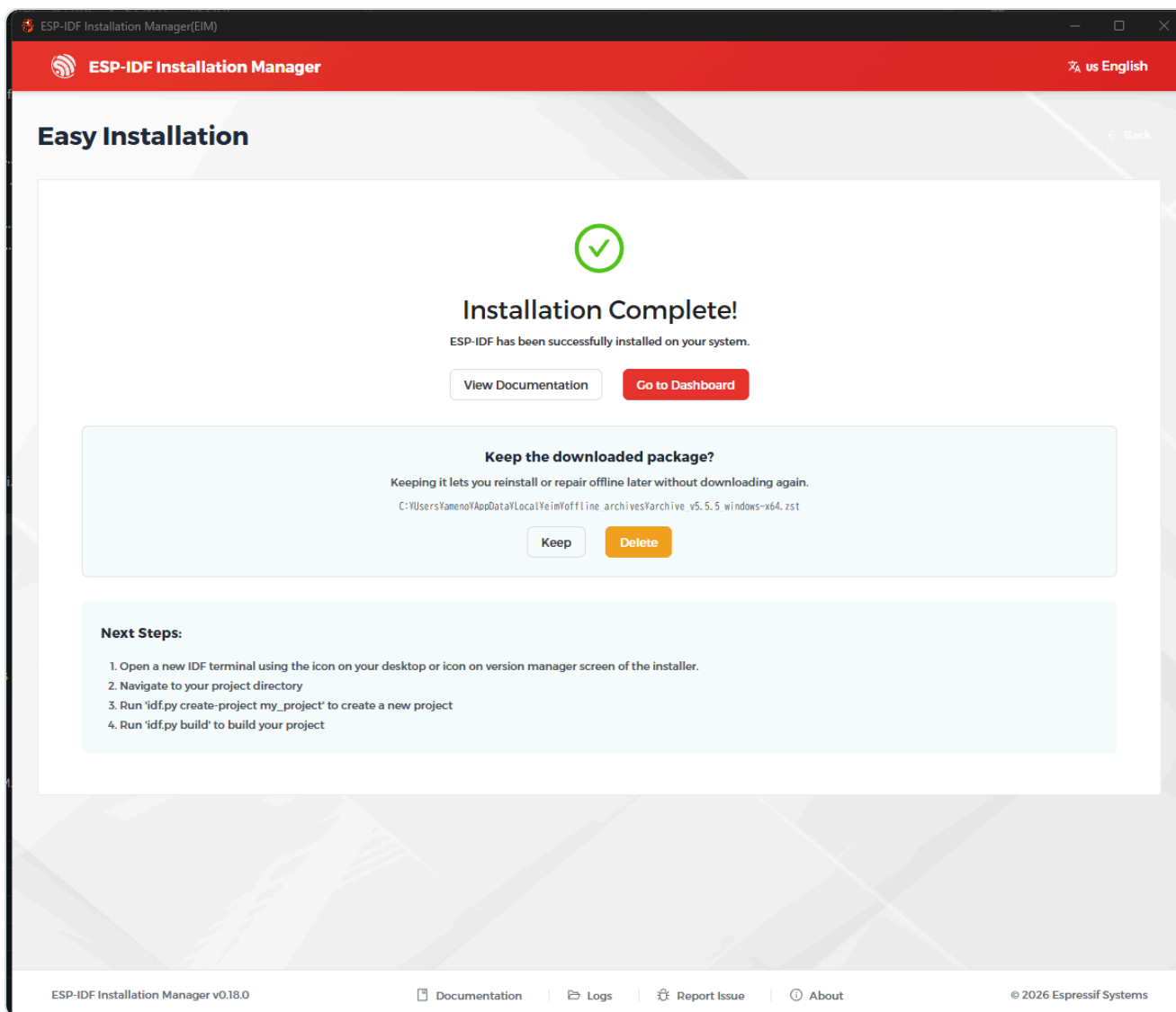


⚠️ 「Installation Complete!」になるまで絶対に閉じないでください

進捗バーが止まって見える、ログの流れが遅くなる、といったことは回線速度によってはよくあります。下の
ような「Installation Complete!」の完了画面が表示されるまでは、途中でウィンドウを閉じたり、PCをス
リープさせたりしないでください。途中で中断すると、次の7章（7-3・7-4）の手順で「ESP-IDFのバージョ
ンが見つからない」「eim_idf.json doesn't exists（eim_idf.jsonが存在しない）」といったエラーの原因にな
ります。45分以上経っても進捗が0%のまま全く動かない場合のみ、いったんウィンドウを閉じ、6-2からや
り直してください。

6-8. インストール完了

すべて完了すると、以下のような「Installation Complete!」画面が表示されます。



- **Keep the downloaded package?** : ダウンロード済みのオフラインパッケージ（`.zst` ファイル）を残すかどうかの確認です。残しておく、万一再インストールが必要になった際に再ダウンロード不要で復旧できます。ディスク容量に余裕があれば「**Keep**」を選んでおくことをおすすめします。
- **Next Steps** : ターミナルから `idf.py` コマンドを直接使う場合の案内です。本書ではVS Code拡張機能からビルド・書き込みを行うため、この案内の手順は使いません。

選択が終わったら、EIMのウィンドウを閉じ（右上の×ボタン）、VS Codeの画面に戻ってください。

7

サンプルプロジェクトの取得とターゲット設定

GET THE SAMPLE PROJECT

7-1. サンプルコードの入手

Elecrow公式GitHubリポジトリから、CrowPanel Advanced 7inch用のサンプルプロジェクト一式を取得します。

```
git clone https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen.git
```

Gitを使わない場合は、リポジトリページ右上の「Code > Download ZIP」から取得し、任意のフォルダに展開してください。 `example` フォルダの中に、ESP-IDF用のサンプルプロジェクトが格納されています。

⚠ Windowsの場合：保存先フォルダのパスは極力短くしてください

このリポジトリはフォルダ名・パス階層が長く（例： `example\V1.1\idf-code\Lesson01-Print_Hello_World` ）、さらにビルド時にはその下へさらに深いフォルダが自動生成されます。 **Windows** には1つのパス全体で260文字までという制限があり、 `ドキュメント\GitHub\...` のような深い場所に保存すると、この上限を超えてビルドが失敗することがあります（8章のビルドで原因不明のCMakeエラーが出た場合、多くはこれが原因です）。

保存先は、 `C:\esp\` のようなドライブ直下に近い短いフォルダにしてください。例えば `git clone` する場所を `C:\esp\CrowPanel` にする、または展開したZIPの中の `example` フォルダだけを `C:\esp\example` のような場所にコピーする、といった対応が有効です。

すでに別の場所で7-2以降を試したあとにフォルダを移動した場合は、移動先のフォルダの中に `build` フォルダや `sdkconfig` ファイルが残っていないか確認し、もしあれば削除してから、7-2～7-5を新しい場所であらためてやり直してください。ESP-IDFバージョンの選択（7-3）やターゲットチップの設定（7-5）はどちらも「開いているフォルダー」ごとに保存される設定のため、フォルダーを変えると引き継がれません。これらをやり直さないままだと、8-4のビルドで「xtensa-esp32-elf-gcc」のような誤ったコンパイラが使われるエラーの原因になります。

7-2. VS Codeでプロジェクトを開く

1. VS Codeのメニューから「ファイル > フォルダーを開く（File > Open Folder）」を選択します。
2. 入手したサンプルプロジェクトのフォルダ（ `CMakeLists.txt` がある階層）を選択します。

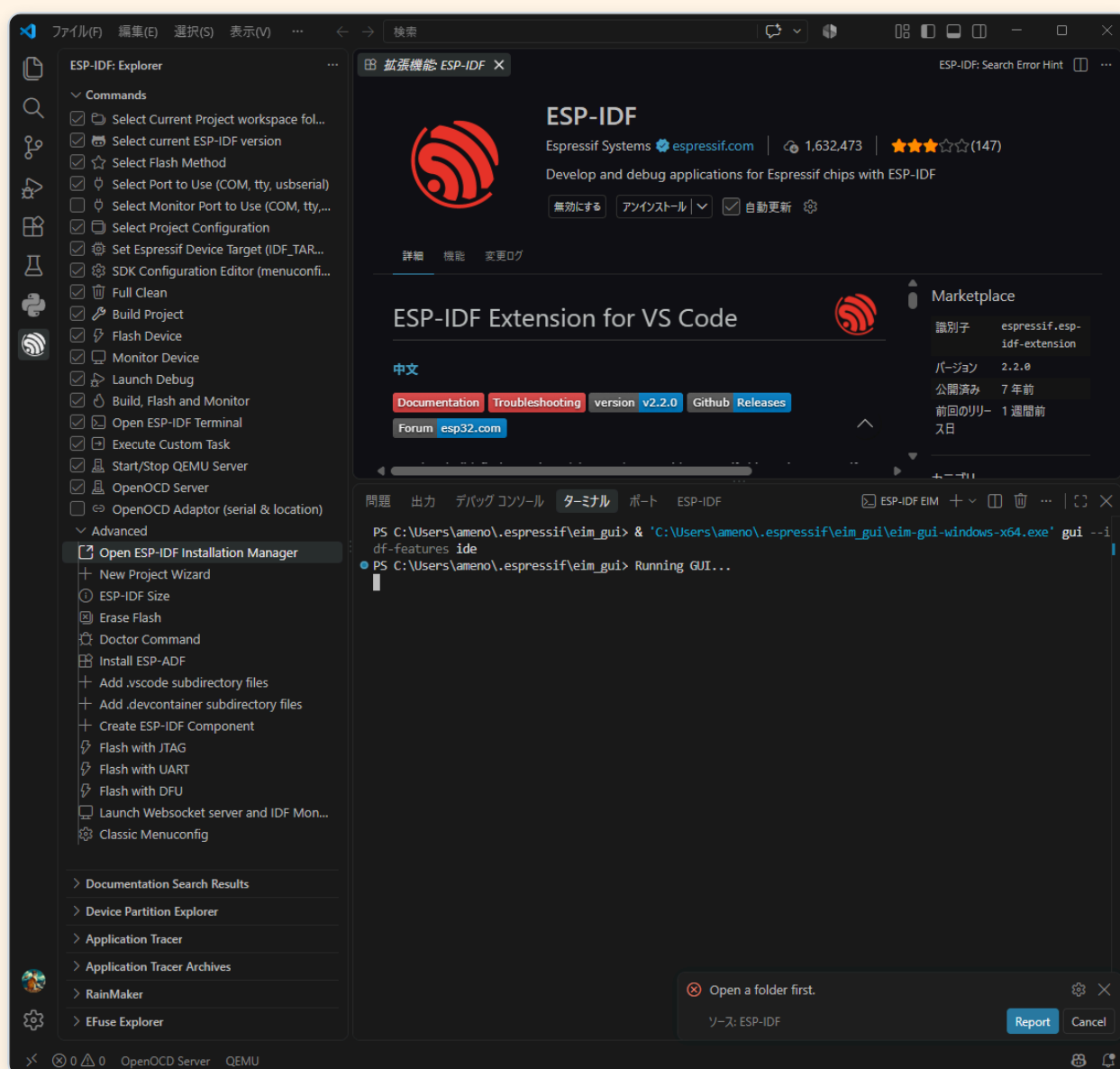
※ここから先の7-3～7-5は、必ずこのフォルダーを開いた状態（VS Codeのウィンドウ左側にファイル一覧が表示されている状態）で行ってください。フォルダーを開く前に実行すると、次の7-3で説明する「Open a folder first」というエラーになります。

7-3. VS Codeに使用するバージョンを教える（Select Current ESP-IDF Version）

6-8までで「ESP-IDF本体」のインストール自体は完了していますが、これだけではVS Codeの拡張機能はまだそれを使ってくれません。「このプロジェクトではどのESP-IDFを使うか」を、拡張機能に明示的に教える必要があります。この一手間を忘れると、後の章でビルドがエラーになったり、次の7-4のDoctor Commandで異常が出たりします。

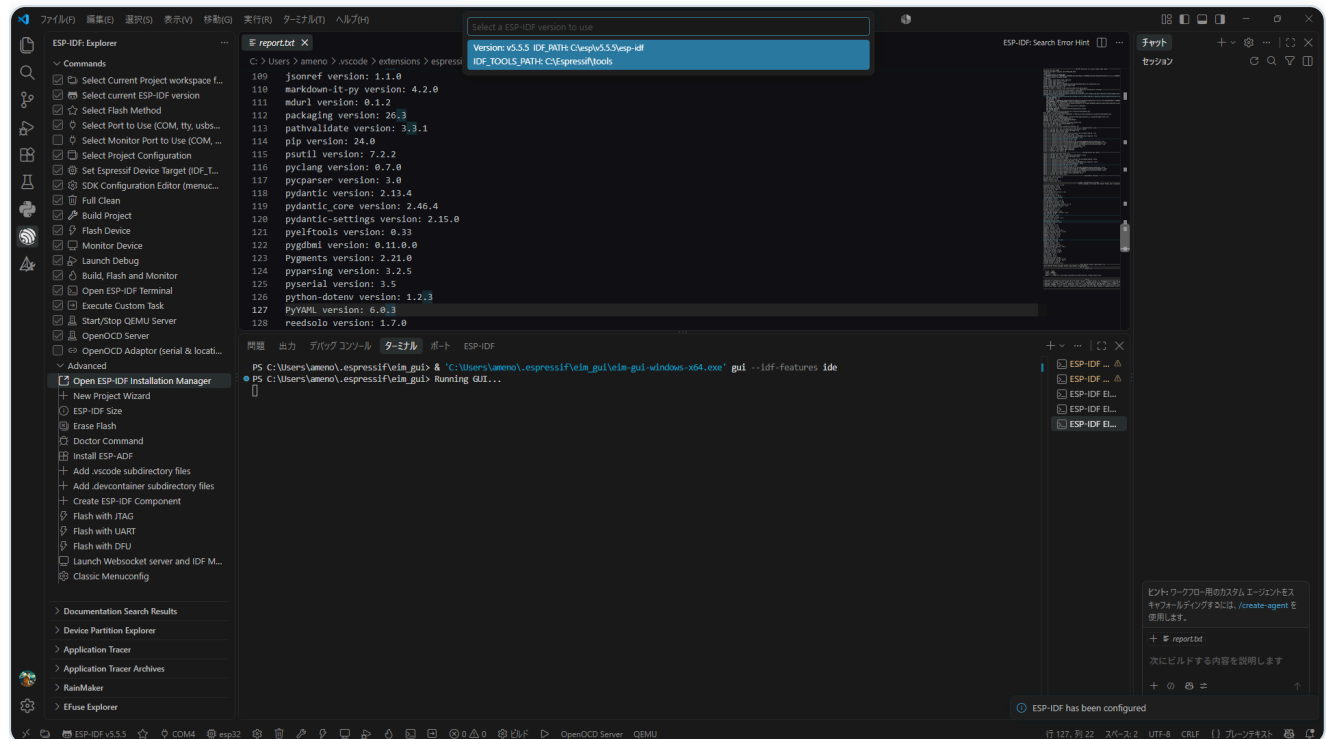
⚠ 先にフォルダーを開いていないと「Open a folder first」エラーになります

この設定はプロジェクトのフォルダーごとに保存される仕組みのため、7-2でフォルダーを開く前にこのコマンドを実行すると、下の画像のように「Open a folder first（先にフォルダーを開いてください）」というエラーが表示され、実行できません。その場合は先に7-2の手順でプロジェクトフォルダーを開いてから、あらためて以下の手順をやり直してください。



1. コマンドパレット（**Ctrl+Shift+P**）から **ESP-IDF: Select Current ESP-IDF Version** を実行します。
2. インストール済みのESP-IDFバージョン一覧が表示されるので、6章で導入したバージョン（本書の例では「**v5.5.5**」）を選択します。

期待される結果：画面下部のステータスバーに、選択したバージョン（例： **ESP-IDF: v5.5.5** ）が表示されます。もし一覧に何も表示されない場合は、6-8のインストールが正常に完了していません。6-2からやり直してください。

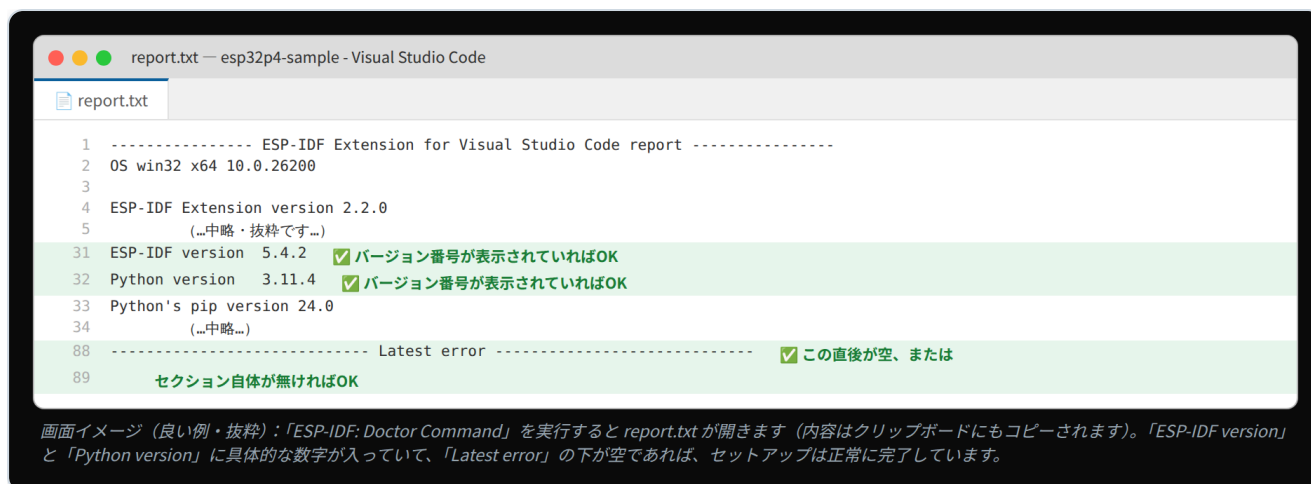


7-4. インストール状態の確認（Doctor Command）

最後に、ここまでの設定（ESP-IDF本体のインストールとバージョン選択）が正しく終わっているかを確認します。

1. コマンドパレットから **ESP-IDF: Doctor Command** を実行します（6-1のExplorerパネルの「Advanced」内からも実行できます）。
2. 実行すると、画面下部にパネルが開くのではなく、「**report.txt**」という診断結果のテキストファイルが **エディタ上に新しく開きます**（内容はクリップボードにもコピーされます）。この中身を上から順に確認します。

期待される結果：report.txtの中の「**ESP-IDF version**」の行に **5.5.5** のような具体的なバージョン番号が、「**Python version**」の行にも具体的な番号が入っており、末尾付近の「**Latest error**」という項目の下が空になっていれば、ESP-IDF本体のセットアップは正常に完了しています。



⚠️ report.txtがこうなっていたら要注意

「ESP-IDF version」が x.x のまま、「Python version」が Not found、末尾付近に「eim_idf.json doesn't exists（存在しない）」というエラーが書かれている場合は、7-3「Select Current ESP-IDF Version」が実行されていない、または6-8のインストールが正常に完了していない状態です。以下の画面イメージは、実際によく起きるこのパターンを示したものです。

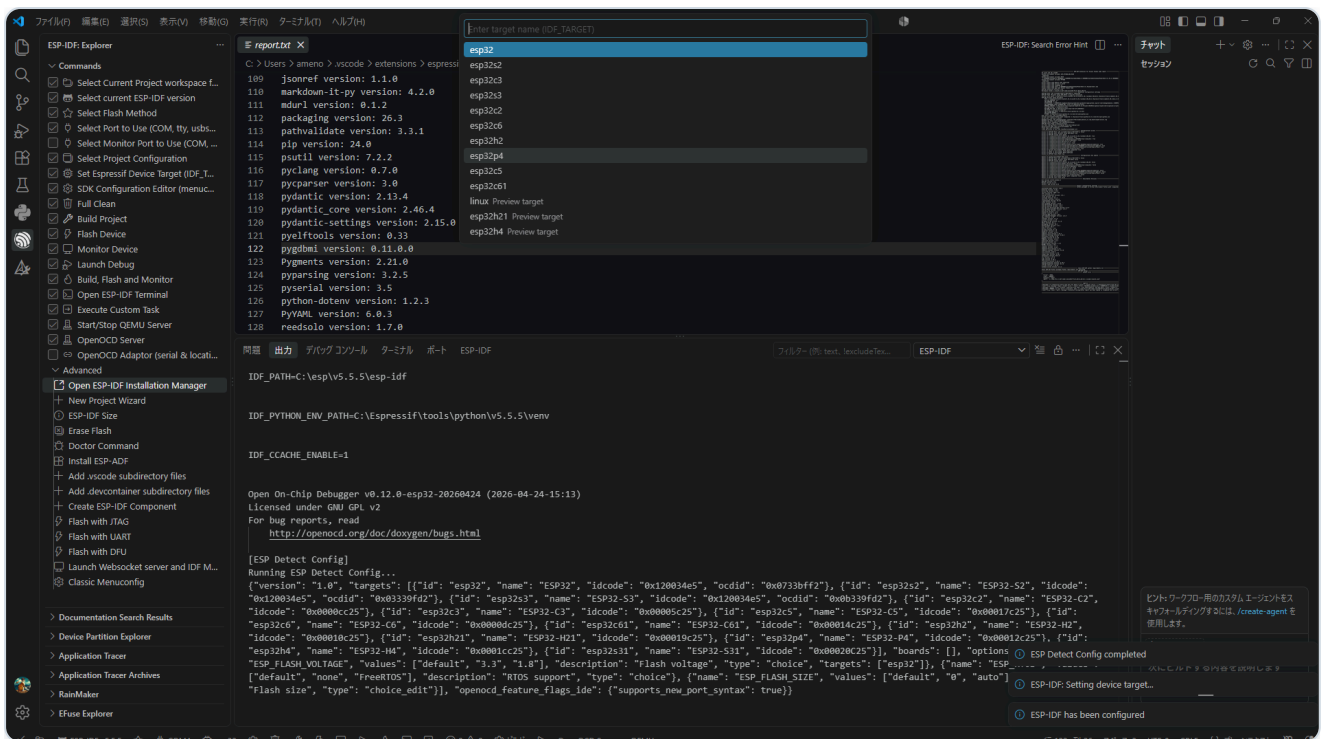


対処方法：まず7-3の手順をもう一度実行し、一覧にバージョンが表示されるか確認してください。一覧に何も表示されない場合は、ESP-IDF本体のインストール自体が完了していないため、6-2からやり直してください（VS Codeを一度再起動してから試すと解消することもあります）。

7-5. ターゲットチップの設定

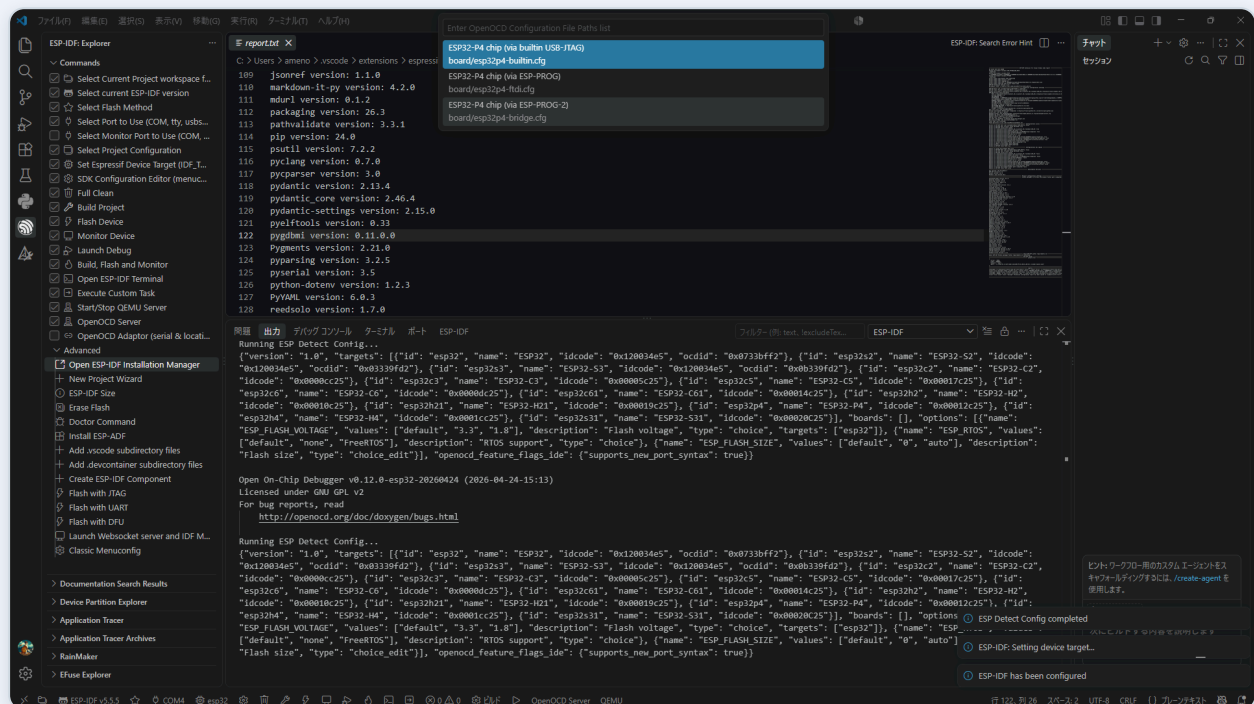
1. コマンドパレットから **ESP-IDF: Set Espressif Device Target** を実行します。
2. 候補一覧から **esp32p4** を選択します。

期待される結果：選択後、画面下部のステータスバーに「esp32p4」と表示されます。以降のビルドはこのチップ向けに行われます。



🔴 続けて「Enter OpenOCD Configuration File Paths list」が出た場合

esp32p4を選択した直後、デバッグ接続方法を尋ねる一覧が続けて表示されることがあります。これは実機デバッグ（JTAG）用の設定で、ビルド・書き込み自体には影響しません。CrowPanel Advanced 7inchはESP-PROGのような外付けデバッガを使わないため、一番上の「ESP32-P4 chip (via builtin USB-JTAG)」(**board/esp32p4-builtin.cfg**) を選択してください。



※ターミナルから直接操作する場合は、以下のコマンドと同等です。

```
idf.py set-target esp32p4
```

7-6. ESP32-P4のチップリビジョン設定（重要・新しいESP-IDFで必須）

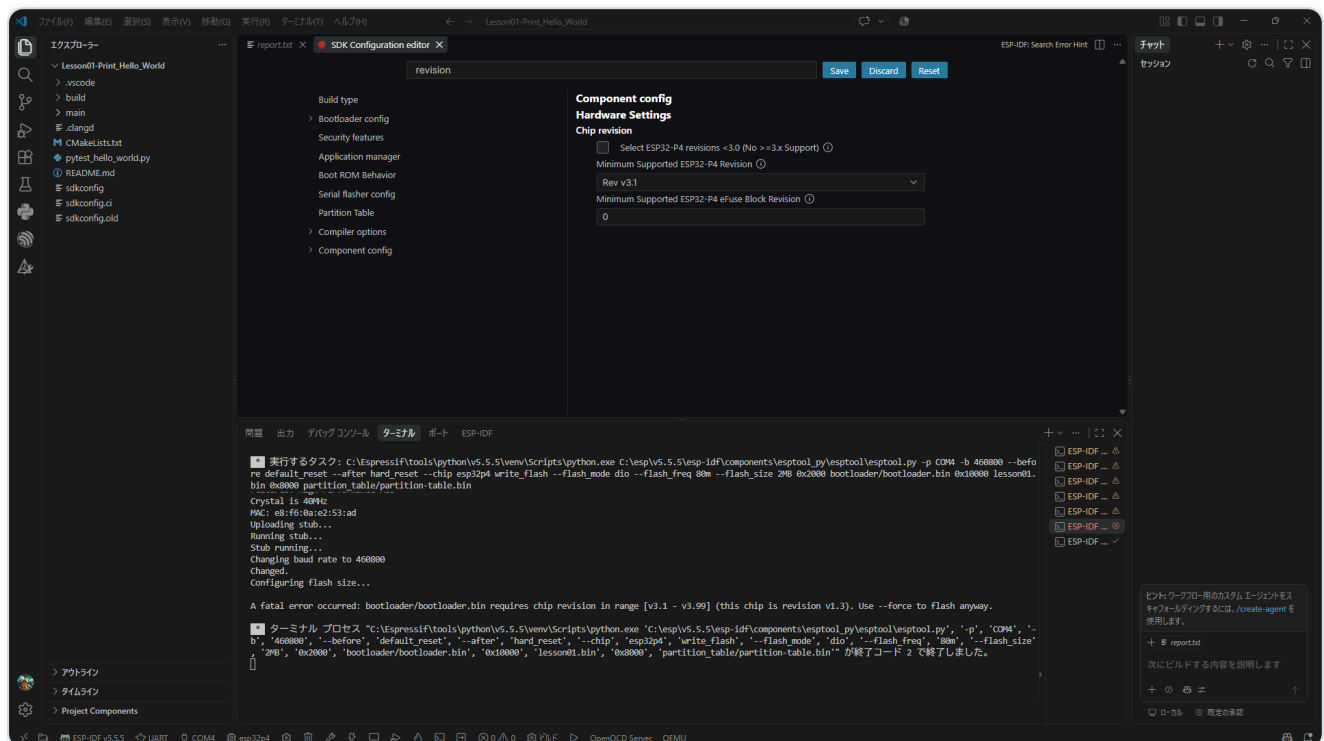
ESP-IDF v5.5系以降では、ビルド設定の既定値が新しいシリコン（チップリビジョンv3.x）を前提にしています。CrowPanel Advanced 7inchに搭載されているESP32-P4はリビジョンv1.3であるため、この設定を行わずにビルド・書き込みを行うと、8-5の書き込み時に必ずエラーになります。先に本手順を済ませてから8章に進んでください。

！ なぜ必要か：ESP32-P4のハードウェア変更

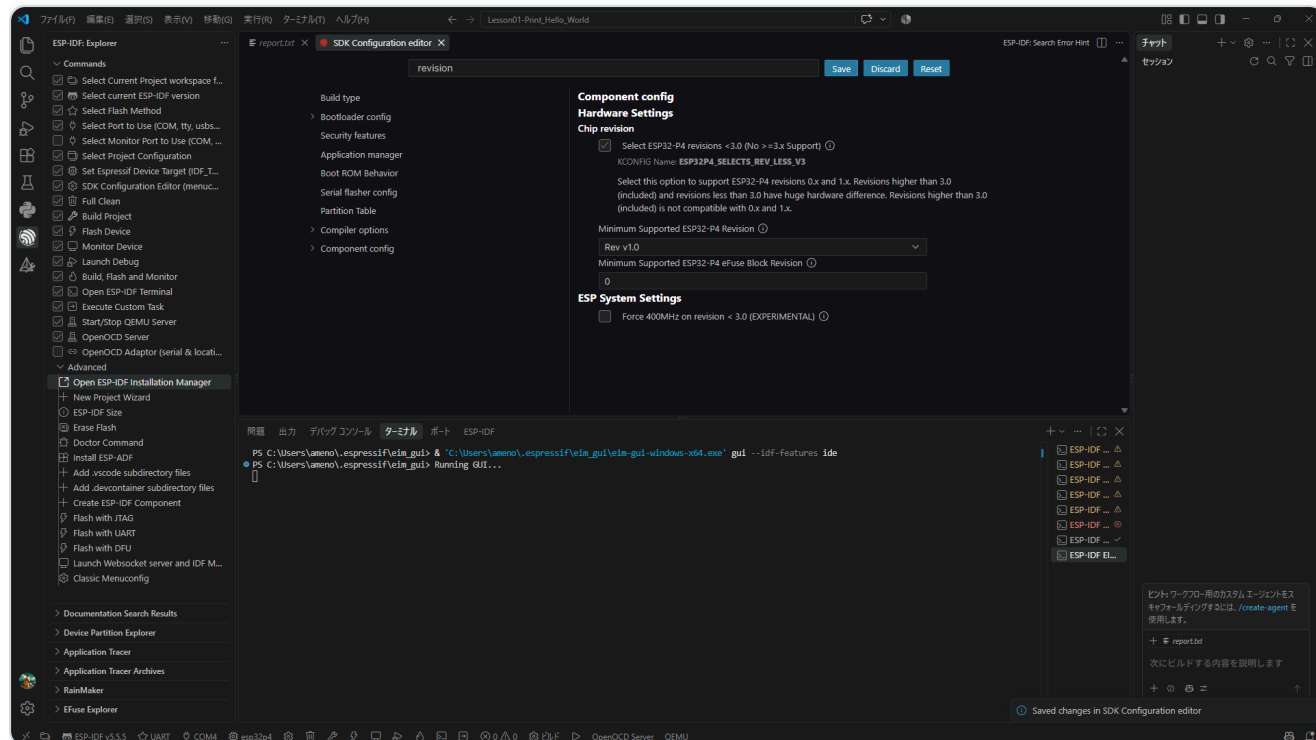
ESP32-P4はチップリビジョン3.0で電源まわり（パワーレール）を含むハードウェア変更が行われており、Espressifはこれをv0.x/v1.x系とは互換性のない世代として扱っています。そのため最近のESP-IDF（本書のv5.5.5を含む）は既定で「Minimum Supported ESP32-P4 Revision」が「Rev v3.1」に設定されており、v1.x世代のチップに対してはビルド成果物の書き込みを拒否します。CrowPanel Advanced 7inchのようにv1.x世代のESP32-P4を搭載した製品では、この設定をv1.0以降に緩める必要があります。

1. コマンドパレットから **ESP-IDF: SDK Configuration Editor**（menuconfigのGUI）を実行します。
2. 開いた画面上部の検索ボックスに **revision** と入力します。
3. 「Component config > Hardware Settings > Chip revision」の下にある「Select ESP32-P4 revisions <3.0 (No >=3.x Support)」(Kconfig名: **ESP32P4_SELECTS_REV_LESS_V3**) のチェックボックスにチェックを入れます。

変更前：チェックが入っておらず、「Minimum Supported ESP32-P4 Revision」は「Rev v3.1」のままです。この状態で書き込みを行うと、後述のようなエラーになります。



変更後：チェックを入れると、「Minimum Supported ESP32-P4 Revision」が自動的に「Rev v1.0」に変わります。画面右上の「**Save**」をクリックして設定を保存してください（画面右下に「Saved changes in SDK Configuration editor」という通知が出れば保存成功です）。



📍 近くにある「Force 400MHz on revision < 3.0 (EXPERIMENTAL)」について

同じ画面の少し下に、実験的機能である「Force 400MHz on revision < 3.0」というチェックボックスもありますが、本書の手順ではチェックを入れません（未チェックのままで問題なく動作します）。

⚠️ 「--force」オプションでの回避はおすすめしません

エラーメッセージには「Use --force to flash anyway.」と表示されますが、この制限はチップリビジョンごとの電源設定・PSRAM/Flashのタイミング等、実際のハードウェアに関わる差異をもとに設けられています。--force で無理に書き込むのではなく、必ず本手順でリビジョン設定を正しく変更してから進めてください。

設定を保存したら、コマンドパレットから **ESP-IDF: Full Clean** を実行し、ビルド成果物を一度すべて削除してから、8章のビルド・書き込みに進んでください（設定変更後は再ビルドが必要です）。

8-1. 本体とPCの接続

1. USB Type-Cケーブルで、CrowPanel Advanced 7inchの「UART0」ポートとPCを接続します。
2. 画面が正常に点灯しない場合は、電源不足の可能性があるので、2章の注意事項を参照し給電を強化してください。

8-2. ダウンロード（書き込み）モードへの切り替え

ESP32-P4を書き込み可能な状態にするため、以下の手順でボタン操作を行います。

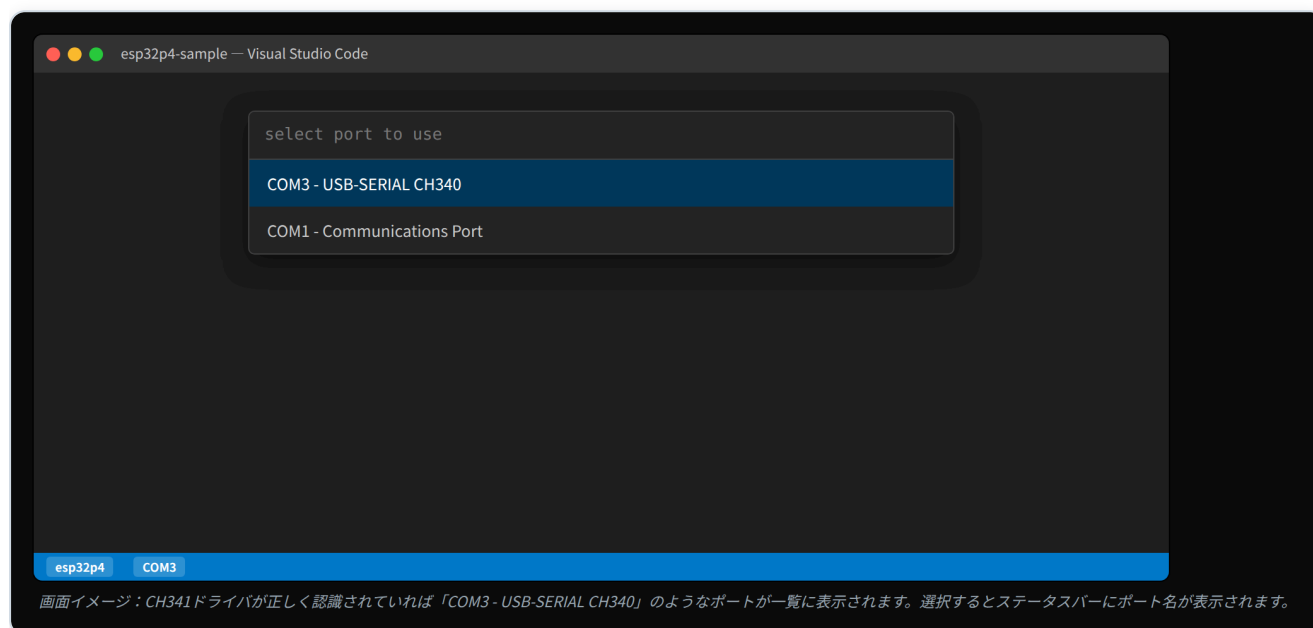
1. **BOOT**ボタンを押したままにする
2. その状態のまま**RESET**ボタンを1回押して離す
3. 最後に**BOOT**ボタンを離す

(BOOTを押しながらRESET → RESETを離す → BOOTを離す、という順番です)

8-3. ポートの選択

1. コマンドパレットから **ESP-IDF: Select Port to Use** を実行します。
2. Windowsでは **COMx**、macOSでは **/dev/tty.wchusbserial*** や **/dev/tty.usbmodem***、Linuxでは **/dev/ttyUSB*** や **/dev/ttyACM*** のような表記のポートが表示されるので、該当するものを選択します。

期待される結果：選択したポートが画面下部のステータスバーに表示されます。一覧に何も表示されない場合は、4章のドライバが正しく認識されているか確認してください。



8-4. ビルド

1. コマンドパレットから **ESP-IDF: Build your Project** を実行します（下部ステータスバーのビルドアイコンからも実行可能）。
2. 初回ビルドは数分かかります。「Build complete」と表示されれば成功です。

期待される結果：下部の「ターミナル」パネルが自動的に開き、緑色の文字で次のように表示されます。赤字の「error」が出ていなければ成功です。



```
esp32p4-sample — Visual Studio Code
問題 ターミナル 出力 デバッグコンソール

[100%] Built target esp32p4-sample.elf
esp32p4-sample.bin binary size 0x1a2f30 bytes
Smallest app partition is 0x300000 bytes. 0x159cd0 bytes (44%) free.
Project build complete. To flash, run this command:
idf.py -p COM3 flash
```

画面イメージ：ビルドが成功すると、ターミナルに緑色で「Project build complete.」と表示されます。エラー（赤字のerror）が出ていないことを確認してください。

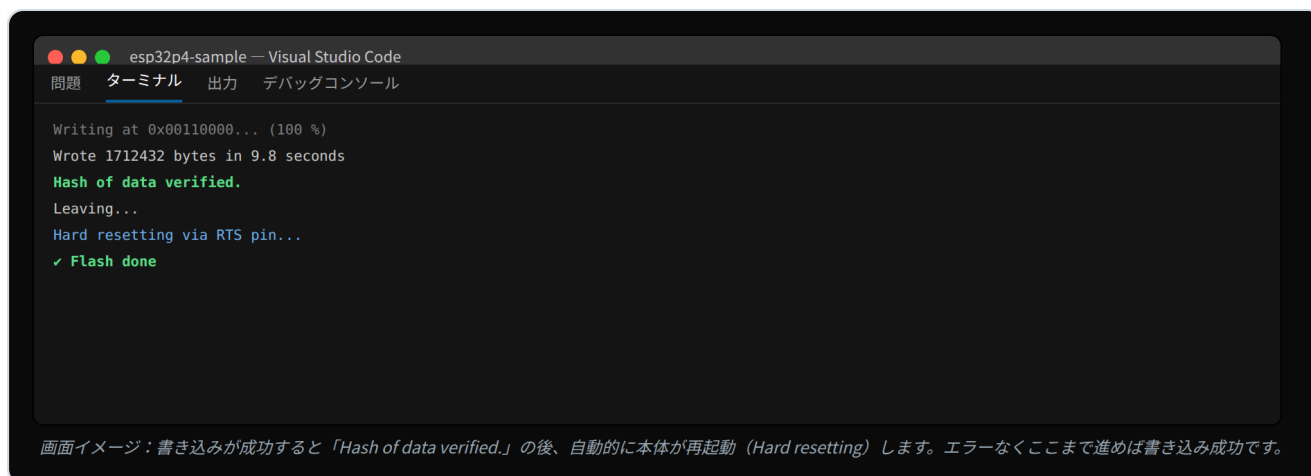
「warning: cannot assign memory region sram_seg ...」が出て問題ありません

ビルドの最後に表示されるメモリ使用量サマリー付近で、上記のような `idf_size.py` 由来の警告が出ることがあります。これはビルド後のサイズ集計ツールの表示上の制約によるもので、Flash／DIRAM／HP-core RAM／LP-RAMなど各領域のサイズが正しく表示され、ビルド自体が「Build complete」で終わっていれば、実害のない警告として無視して問題ありません。

8-5. 書き込み（Flash）

1. 8-2の手順でダウンロードモードにした状態のまま、コマンドパレットから **ESP-IDF: Flash your Project** を実行します。
2. 書き込み方式は「UART」を選択します。
3. 進捗が100%になり、エラーなく完了すれば書き込み成功です。

期待される結果：ターミナルに次のように表示され、「Hash of data verified.」の後に本体が自動的に再起動（Hard resetting）します。



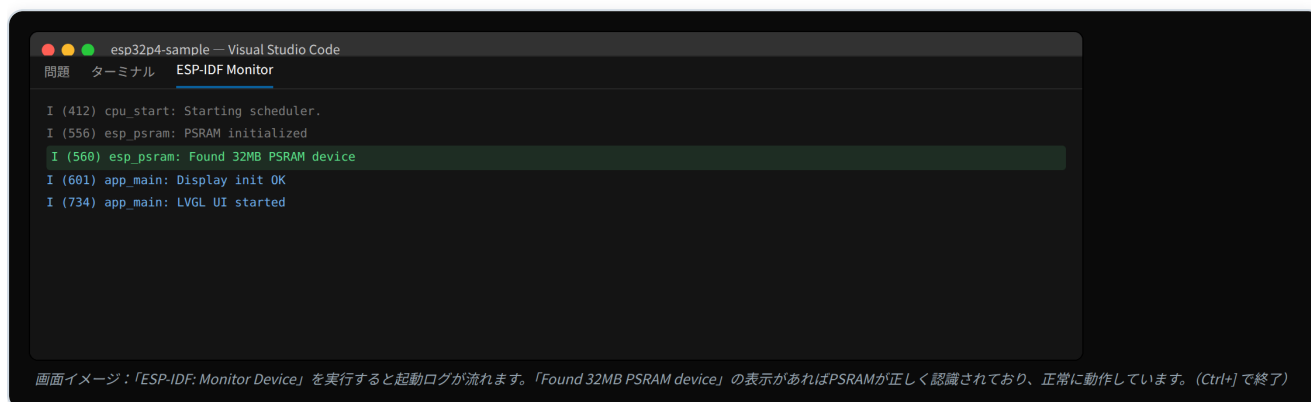
⚠️ 「requires chip revision in range [v3.1 - v3.99] (this chip is revision v1.3)」というエラーが出た場合

7-6のチップリビジョン設定がまだ行われていません。7-6の手順に戻って設定を行い、保存後に **ESP-IDF: Full Clean** を実行してから、あらためて8-4のビルド→8-5の書き込みをやり直してください。

8-6. シリアルモニタでの動作確認

1. 本体の**RESET**ボタンを押して再起動します（通常起動に戻ります）。
2. コマンドパレットから **ESP-IDF: Monitor Device** を実行し、シリアルログを表示します。
3. ログ中に **Found 32MB PSRAM device** のような表示があれば、PSRAMが正しく認識されており正常です。
4. 本体の画面にサンプルの表示（LVGLのUI等）が出ていれば、セットアップは完了です。

期待される結果：ターミナルに起動ログが流れ続けます。次のように「Found 32MB PSRAM device」の行が表示されていれば正常です（終了するには **Ctrl+]**）。



✅ ここまでできれば開発環境構築は完了です

以降は **example** フォルダ内の各サンプル（タッチ操作、Wi-Fi、カメラ、LVGL UI等）を開いて同様にビルド・書き込みすることで、CrowPanel Advanced 7inchの各機能を試すことができます。

症状	主な原因・対処法
8-5の書き込み時に 「bootloader/bootloader.bin requires chip revision in range [v3.1 - v3.99] (this chip is revision v1.3). Use --force to flash anyway.」 というエラーが出る	7-6のチップリビジョン設定がまだ行われていません。 7-6の手順に従い、SDK Configuration Editorで「Select ESP32-P4 revisions <3.0 (No >=3.x Support)」にチェックを入れて保存し、 ESP-IDF: Full Clean を実行してから、8-4のビルド→8-5の書き込みをやり直してください。エラーメッセージ中の --force は使わないでください（7-6参照）。
画面が真っ暗、または表示が乱れる	ほとんどのケースはPSRAM未検出が原因です。シリアルモニタのログでPSRAM検出メッセージを確認してください。給電不足が原因のこともあるため、ACアダプタでの給電も試してください。
「ESP-IDF: Select Port to Use」で 「No serial ports found」となり、シリアルポートが一覧に表示されない	次を順番に確認してください。①本体の2つあるUSB-CポートのうちUART0（CH341経由の方。USB 2.0 Deviceではない）に接続しているか。②USBケーブルが充電専用ではなくデータ通信対応か（別のケーブルで試す）。③PC側のUSBポートを変えてみる（USBハブは避け、PC本体のポートに直接挿す）。④4章の手順に沿ってCH341ドライバが入っているか、Windowsならデバイスマネージャーで「USB-SERIAL CH340」がポート一覧に表示されるか、macOSなら4章記載のmacOS既知の不具合と対処（「USB 2.0 Device」ポートへの一時的な切り替え等）を確認する。⑤それでも表示されない場合はVS Codeを再起動してから再度試してください。
書き込み（Flash）が失敗する	ダウンロードモードに正しく入れていない可能性があります。8-2のBOOT/RESETボタン操作を再度行ってから書き込みを実行してください。チップリビジョン関連のエラーが出ている場合は、上の行と7-6を参照してください。
ビルド時に型名・API関連のエラーが出る （例： lcd_color_rgb_pixel_format_tが見 つからない等）	基本的なHello Worldサンプルは本書のv5.5.5で動作確認済みですが、ディスプレイ（LVGL）まわりの一部サンプルは、ESP-IDFのバージョンによってAPI名称が変更されている場合があります（例： lcd_color_rgb_pixel_format_t → lcd_color_format_t ）。サンプルコード側のAPI呼び出しを使用中のESP-IDFバージョンに合わせて修正するか、Elecrow公式が前提とする v5.4.2 を導入したい場合は10-3（付録）のCustom Installationの手順を参照してください。
ESP-IDF Installation Managerのダウンロードが途中で止まる	ネットワーク環境（社内プロキシ・VPN等）が影響している場合があります。別のネットワーク環境で再試行するか、ダウンロード元をGitHub／Espressifで切り替えて試してください。
Doctor Commandのreport.txtで 「ESP-IDF version」が「x.x」の	ESP-IDF本体は入っていても、VS Codeにどのバージョンを使うか教える手順（7-3「ESP-IDF: Select Current ESP-IDF Version」）がまだ実行されていない

まま、「Python version」が「Not found」、または「eim_idf.json doesn't exists」というエラーが出る	状態です。まず7-3を実行し、バージョン一覧が表示されるか確認してください。一覧が空の場合は6-8のインストール自体が完了していないため、6-2からやり直してください。
ESP-IDF: Select Current ESP-IDF Version を実行すると「Open a folder first」というエラーが出る	プロジェクトのフォルダーを開く前にこのコマンドを実行しています。この設定は開いているフォルダーごとに保存されるため、先に7-2の手順でプロジェクトフォルダーを開いてから、あらためて7-3を実行してください。
ESP-IDF: Select Current ESP-IDF Version を実行しても一覧に何も表示されない	6-8のインストールが正常に完了していません。VS Codeを再起動してから再度お試しください。6-2「Open ESP-IDF Installation Manager」からインストールをやり直してください。
7-5のターゲット設定や8-4のビルド時に 「CMake Error ... Failed to generate test project build system」 「CMakeDetermineCompilerABI」のようなエラーが出る（主にWindows）	ほとんどの場合、プロジェクトの保存場所のパスが長すぎるのが原因です（Windowsは1つのパス全体で260文字までという制限があります）。プロジェクトフォルダを C:\esp\ のようなドライブ直下に近い短い場所に移動し、VS Codeでそのフォルダを開き直してから、7-3～7-5のターゲット設定をやり直してください。詳しくは7-1の注意事項を参照してください。
8-4のビルド時に 「The CMAKE_C_COMPILER: xtensa-esp32-elf-gcc is not a full path and was not found in the PATH」のようなエラーが出る	ターゲットチップがesp32p4（RISC-V用ツールチェーン）ではなく、無印 esp32（Xtensa用ツールチェーン）のまま設定されています。多くは、プロジェクトフォルダを別の場所に移動・コピーした後に7-5をやり直していないことが原因です。現在開いているフォルダの中の build フォルダと sdkconfig ファイルを削除し、あらためてコマンドパレットから「ESP-IDF: Set Espressif Device Target」を実行して esp32p4 を選び直してから、再度ビルドしてください。
7-5のターゲット設定や8-4のビルド時に 「riscv32-esp-elf-gcc.exe: fatal error: cannot execute 'cc1': CreateProcess: No such file or directory」のようなエラーが出る（Windows）	本書執筆時点で、この不具合はCustom Installationを使った場合にのみ確認されています。本書のメイン手順（6章）で案内しているEasy Installationは、あらかじめ動作確認済みの一式をまとめて展開する方式のため、この問題は発生していません。もし10-3（付録）のCustom Installationを使っていてこのエラーが出た場合は、以下を確認してください。 riscv32-esp-elf ツールチェーンの展開（解凍）が不完全な状態です。実際に発生した例では、C:\Espressif\tools\riscv32-esp-elf\ 以下の bin フォルダ（riscv32-esp-elf-gcc.exe など）だけは存在するのに、同じ階層にあるはずの lib ・ libexec ・ include ・ picolibc ・ share フォルダの中身が空になっており、cc1.exe（本来 libexec\gcc\riscv32-esp-elf\...\cc1.exe にある）ごと丸ごと欠けていました。 セキュリティソフトによる誤検知・隔離が原因ではないケースがあることを確認済みです。あるケースでは、隔離履歴も一切なく、Custom Installation

でのインストールを2回やり直しても同じ箇所が同じように空になりました。
一方、ダウンロード済みのZIPファイル
(`C:\Espressif\dist\riscv32-esp-elf-...-x86_64-w64-mingw32.zip`) 自体は7-Zipの「テスト」機能でエラーなし、7-Zipで手動展開すると正常にすべてのファイルが展開できました。つまり**ダウンロードしたファイルは壊れておらず、Custom Installationの展開処理がこのアーカイブをうまく解凍できていない**、という状態が確認されています。

確認方法：エクスプローラーで `C:\Espressif\tools\riscv32-esp-elf\` (バージョン名) `\riscv32-esp-elf\libexec` フォルダを開き、中身が空になっていないか確認してください。

対処方法：①最も簡単な回避策は、6章のEasy Installationに切り替えてやり直すことです (7-6のチップリビジョン設定と併用可能です)。②どうしても特定バージョン固定のCustom Installationを使い続ける必要がある場合は、
`C:\Espressif\tools\riscv32-esp-elf` フォルダを丸ごと削除し、7-Zip等の解凍ソフトで `C:\Espressif\dist` フォルダ内の `riscv32-esp-elf-...-x86_64-w64-mingw32.zip` を右クリックして「7-Zip>テスト」でファイル自体が壊れていないか確認し、問題なければ同じく右クリックから「7-Zip>展開先を指定...」で展開先を
`C:\Espressif\tools\riscv32-esp-elf\esp-14.2.0_20241119\` (バージョン部分はお使いの環境に合わせてください) に直接指定して手動展開してください (EIMが作った既存のフォルダと重なる場合は上書き)。展開後、`libexec` フォルダの中に `cc1.exe` が存在することを目視確認してから次に進んでください。

本体が繰り返し再起動する	電源不足が主な原因です。USB給電ではなく、別電源のACアダプタからの給電に切り替えてください。
--------------	--

10-1. よく使うVS Codeコマンド一覧

コマンドパレットでの名称	役割	ショートカット (Win/Linux)
ESP-IDF: Open ESP-IDF Installation Manager	ESP-IDF本体・ツール一式のインストーラを起動	—
ESP-IDF: Select Current ESP-IDF Version	使用するESP-IDFバージョンをVS Codeに設定	—
ESP-IDF: Doctor Command	環境診断・不具合レポート出力 (report.txtを開く)	—
ESP-IDF: Set Espressif Device Target	対象チップ (esp32p4等) の設定	—
ESP-IDF: Select Port to Use	接続するシリアルポートの選択	—
ESP-IDF: SDK Configuration Editor	menuconfig (sdkconfig) のGUI編集	—
ESP-IDF: Build your Project	プロジェクトのビルド	Ctrl+E B
ESP-IDF: Flash your Project	デバイスへの書き込み	Ctrl+E F
ESP-IDF: Monitor Device	シリアルモニタの起動	Ctrl+E M
ESP-IDF: Full Clean	ビルド成果物を全て削除	—

10-2. 参考リンク

- Elecrow公式コース資料 (Advance HMI P4 7inch Course V1.1) :
https://www.elecrow.com/download/product/DHE04107D/Advance_HMI_P4_7inch_Course_V1.1.pdf
- Elecrow公式Wiki (CrowPanel Advanced 7inch) :
<https://www.elecrow.com/wiki/>
- Elecrow公式GitHubリポジトリ :
<https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen>
- ESP-IDF VS Code拡張機能 公式ドキュメント :
<https://docs.espressif.com/projects/vscode-esp-idf-extension/>

- ESP-IDF Installation Manager 公式ドキュメント：
<https://docs.espressif.com/projects/idf-im-ui/>
- ESP32-P4 Get Started (ESP-IDF公式)：
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32p4/get-started/index.html>
- WCH CH341ドライバダウンロード (Windows / macOS / Linux)：
https://www.wch-ic.com/downloads/CH341SER_ZIP.html / [CH34XSER_MAC_ZIP.html](https://www.wch-ic.com/downloads/CH34XSER_MAC_ZIP.html) / [CH341SER_LINUX_ZIP.html](https://www.wch-ic.com/downloads/CH341SER_LINUX_ZIP.html)

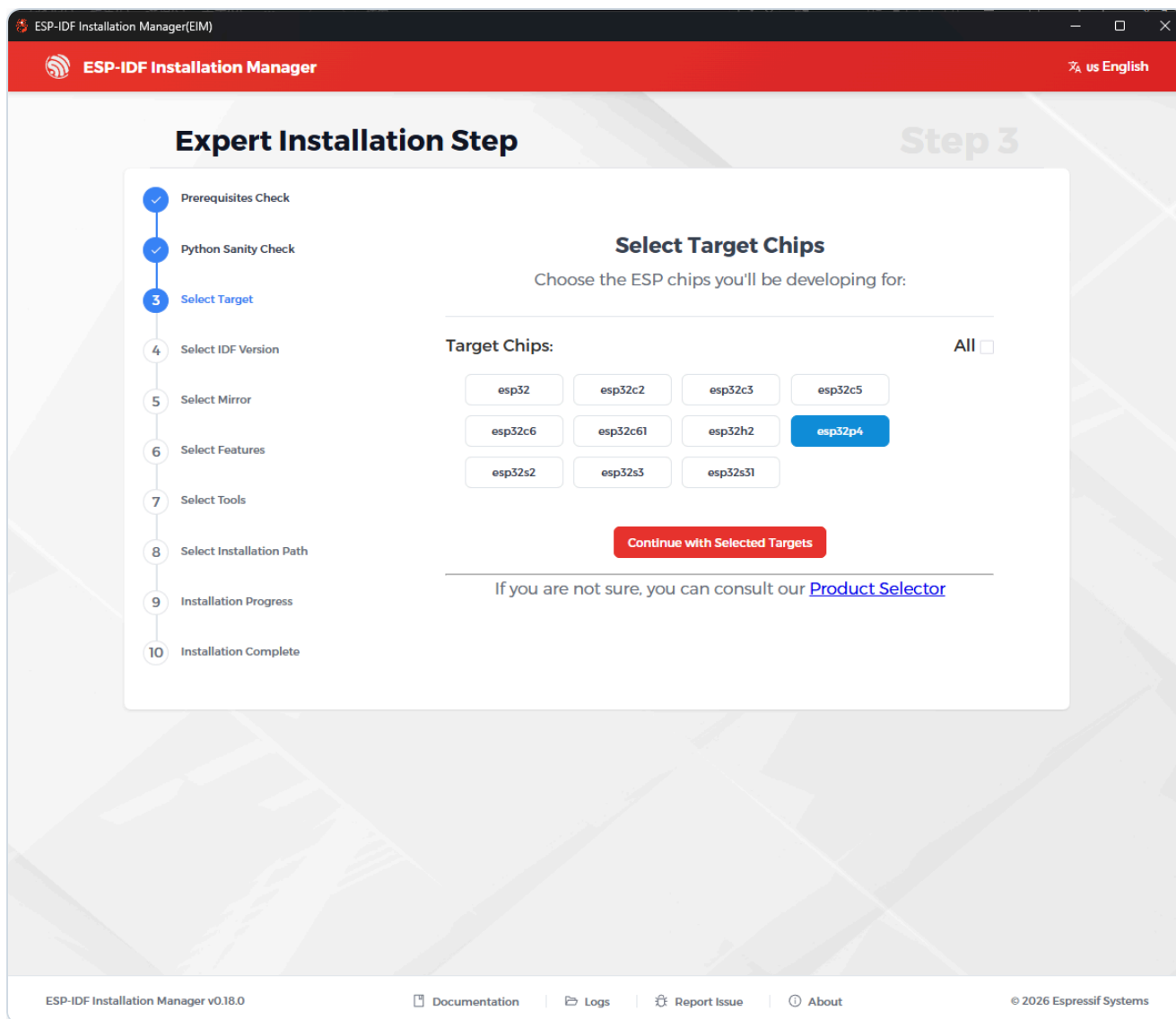
10-3. 付録：特定のESP-IDFバージョンを指定したい場合 (Custom Installation)

6章で案内したEasy Installationは、既定で最新の安定版ESP-IDFを導入します。Elecrow公式サンプルが前提とする **v5.4.2** のように、あえて特定の古いバージョンを指定したい場合は、6-5の画面で「Easy Installation」の代わりに「**Custom Installation**」の「**Start Configuration Wizard**」を選ぶことで、バージョンを含めて細かく指定できます。以下はその手順です（6-4のウェルカム画面までは共通です）。

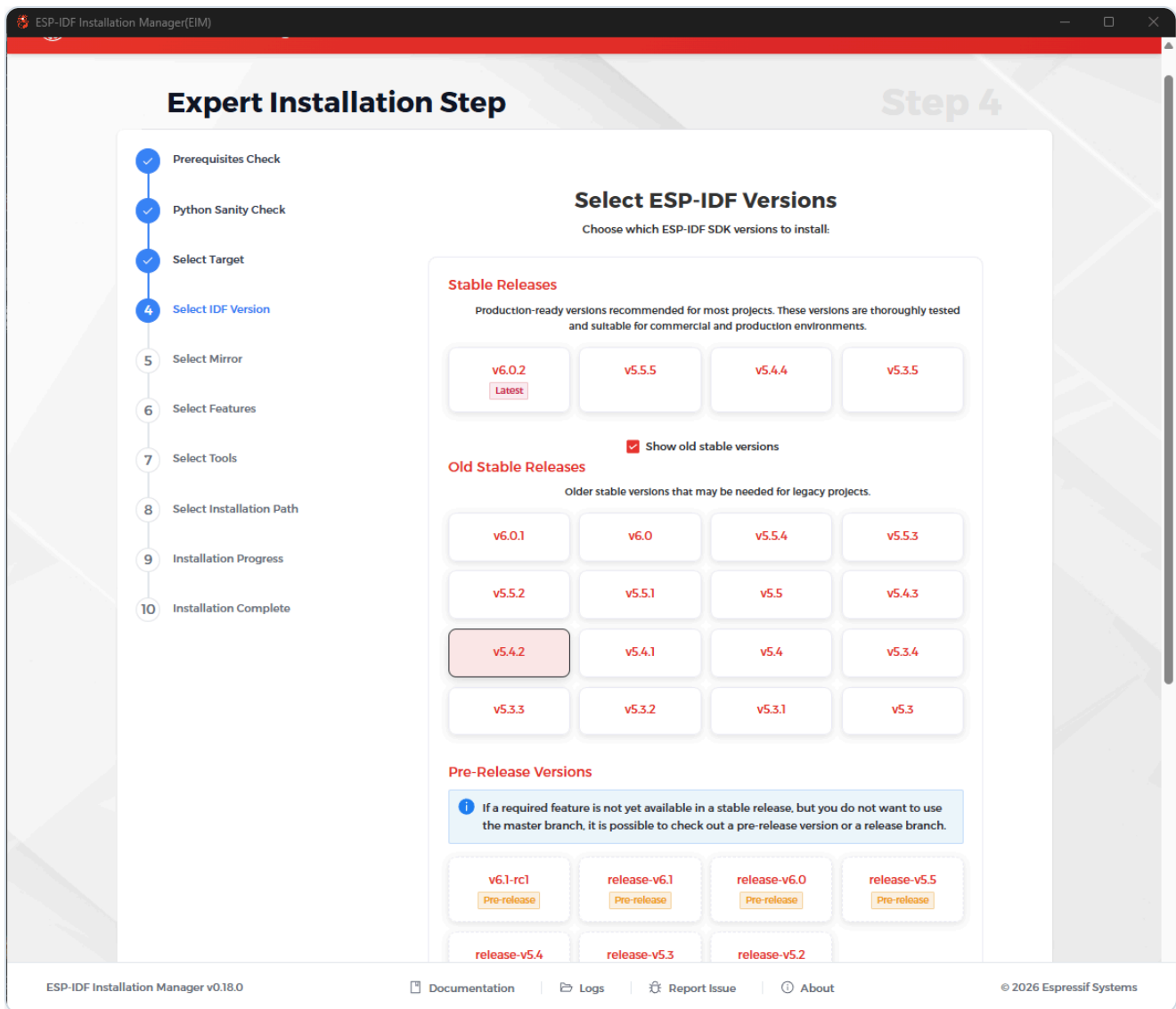
⚠ Custom Installationでは既知の展開エラーが起きる場合があります

本書の検証では、Custom Installationで **riscv32-esp-elf** ツールチェーンの展開（解凍）が不完全になり、ビルド・書き込み時に「cannot execute 'cc1」というエラーが出る不具合を確認しています（9章のトラブルシューティング参照）。この問題はダウンロードファイル自体の破損ではなく、Custom Installationの展開処理側の問題と考えられます。特定バージョンの固定がどうしても必要な場合を除き、まずは6章のEasy Installationを使うことを推奨します。

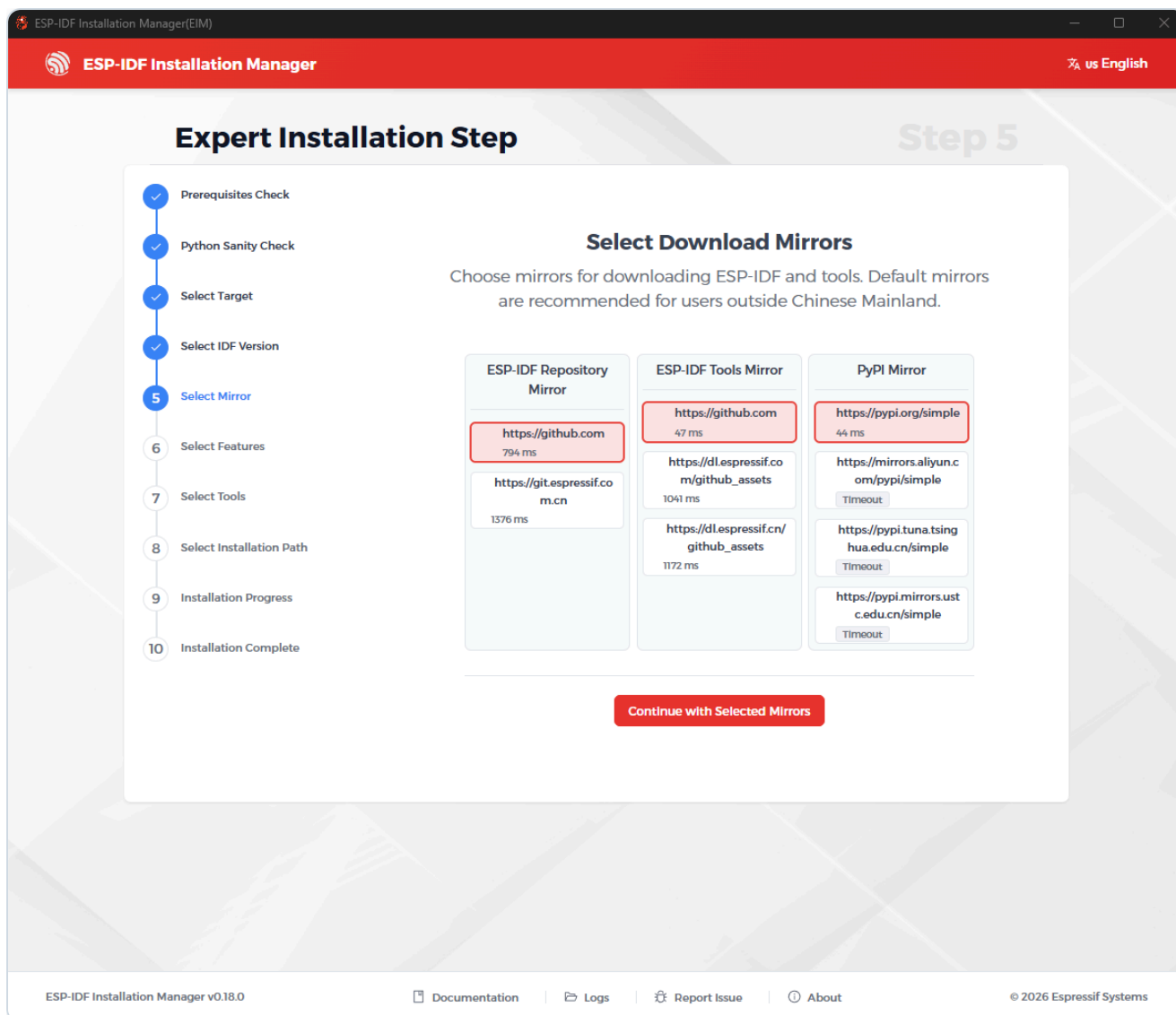
1. 対象チップの一覧から「**esp32p4**」をクリックして選択し（青色にハイライトされます）、「**Continue with Selected Targets**」をクリックします。



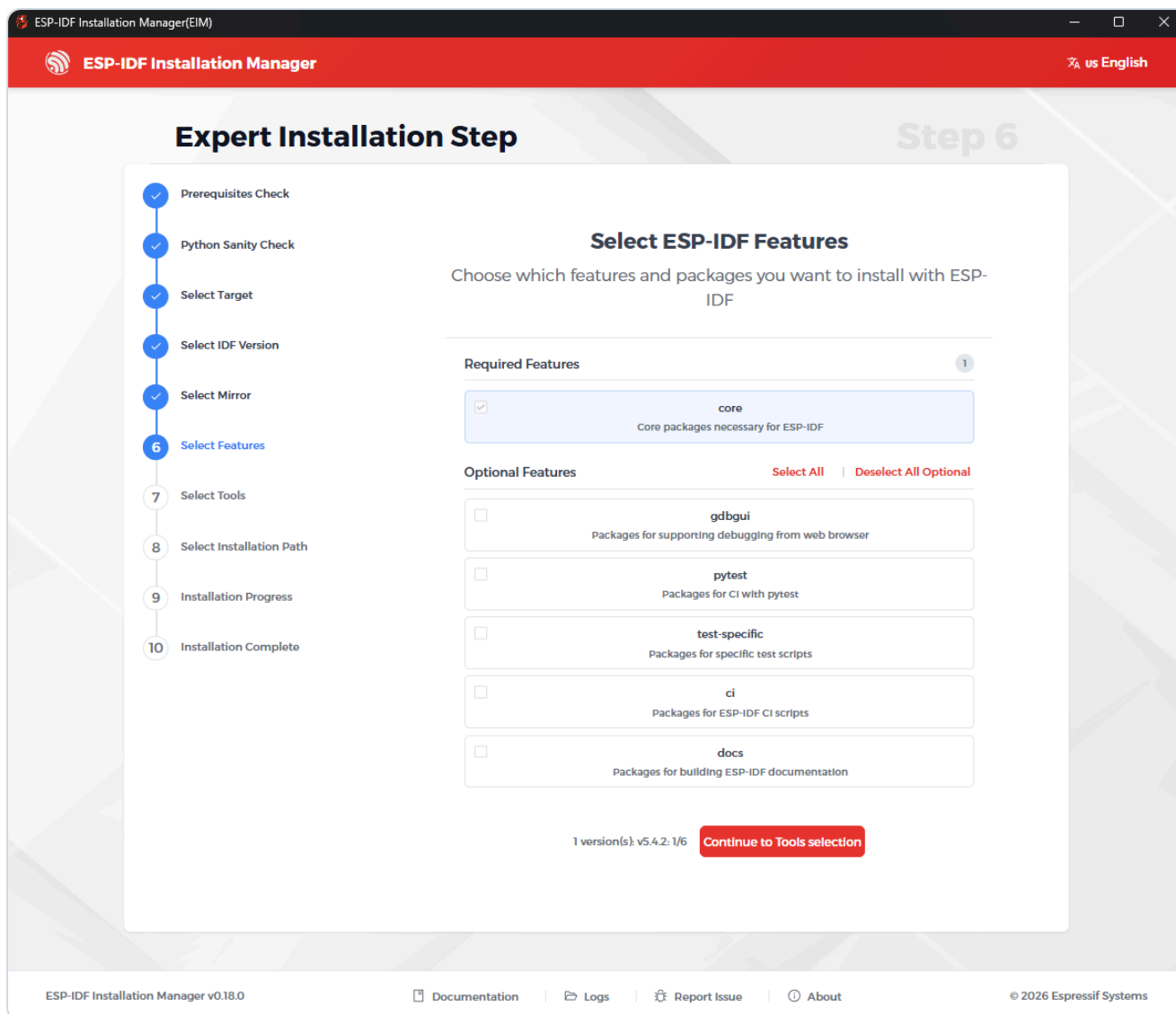
2. ESP-IDFバージョンの選択画面で、既定では最新の安定版（Stable Releases）だけが表示されています。 **v5.4.2** のような古いバージョンは「Old Stable Releases」に含まれるため、まず「**Show old stable versions**」にチェックを入れてください。表示された一覧から目的のバージョン（例：**v5.4.2**）をクリックして選択し、画面下部の「Continue」をクリックします。



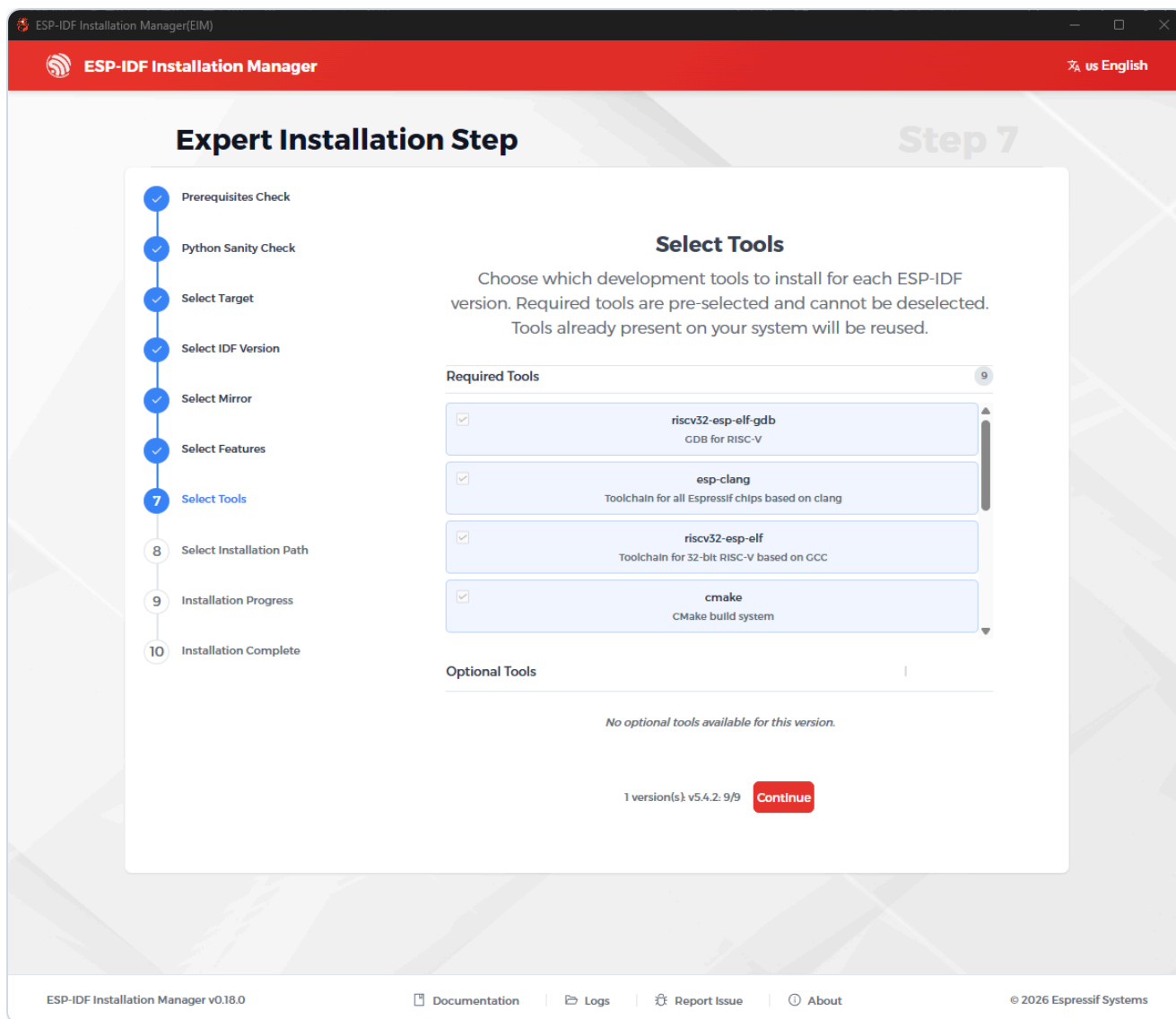
3. ダウンロードミラーの選択画面が表示されます。これは6-3で選んだ「EIM自身の入手元」とは別に、今度は「ESP-IDF本体をどこから取得するか」を選ぶものです。各列の一番上（GitHub / github.com / pypi.org）が既定で選択されているので、通常は変更せずそのまま「**Continue with Selected Mirrors**」をクリックしてください。



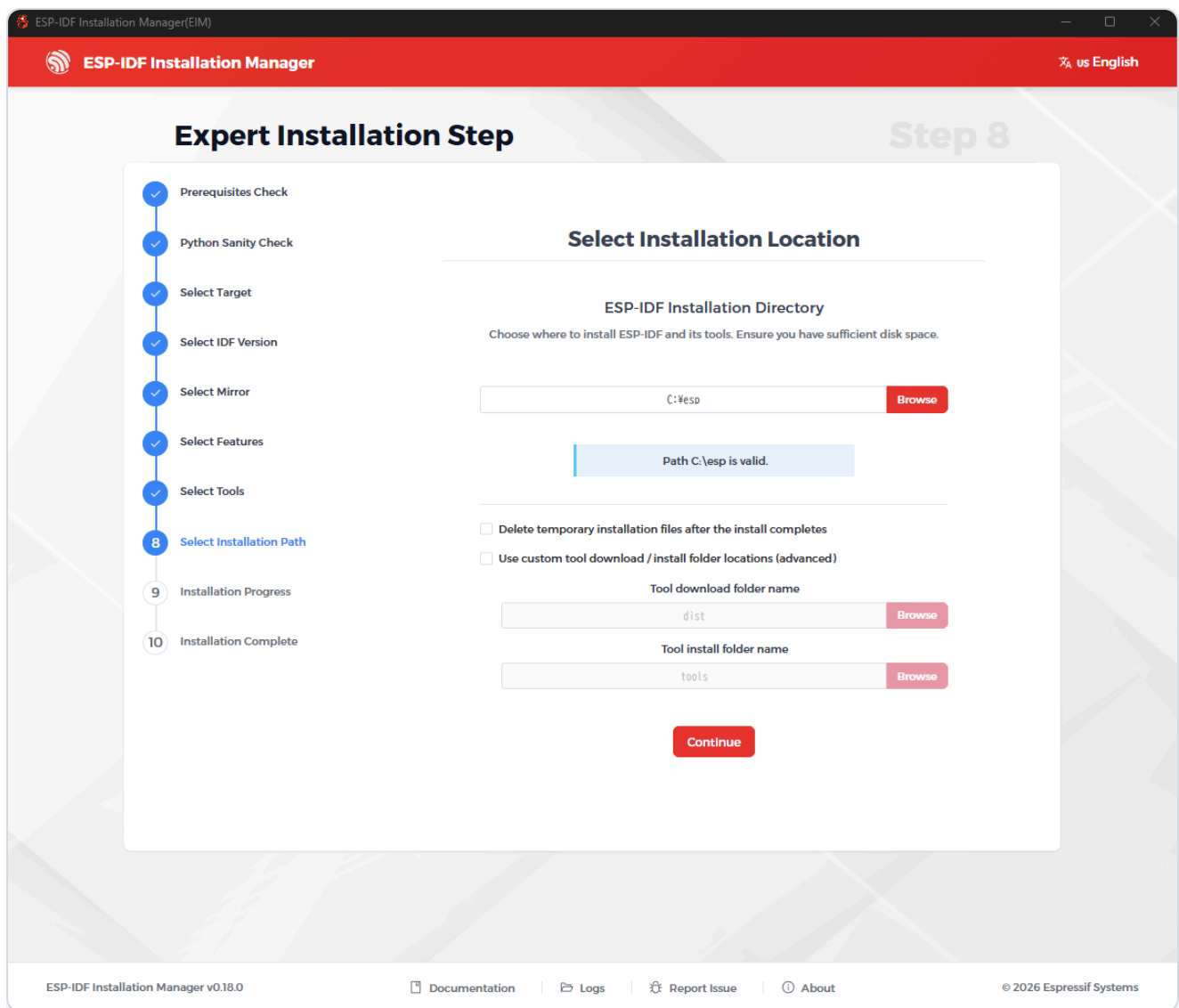
4. インストールする機能の選択画面が表示されます。「core」（ESP-IDF本体に必須の機能）は最初からチェックされており、外すことはできません。「gdbgui」「pytest」等の追加機能は本書の内容では不要なので、チェックを入れずそのまま「**Continue to Tools selection**」をクリックしてください。



5. インストールするツールの一覧が表示されます。すべてチェック済みで変更できません。内容を確認したら「**Continue**」をクリックします。



6. インストール先フォルダの選択画面が表示されます。「ESP-IDF Installation Directory」欄で、既定の `C:\esp` のような短いパスのままで問題ありません。「Delete temporary installation files after the install completes」「Use custom tool download/install folder locations」はどちらもチェックせず、「Continue」をクリックしてください。



以降は6-7・6-8と同様に「Start Installation」をクリックし、インストール完了まで待ちます。完了後は本書の7章以降を同様に進めてください（7-3で選ぶバージョンは、ここで指定した特定バージョンになります）。

本書はElecrow社公式資料および各種公式ドキュメントを参考に作成した非公式の手順書です。製品仕様やソフトウェアのバージョンアップにより、実際の画面表示やメニュー名が変更される場合があります。最新情報は各公式サイトをご確認ください。